

NetSentinel AI Lab

**Explainable Network Anomaly Detection
and Traffic Simulation**

A Project Report

Submitted in partial fulfilment of the requirements
for the degree of

Master of Science in Computer Science

Submitted by
[Student Name]
[Register Number]

Under the guidance of
[Supervisor Name and Designation]

[Department Name]
[College or Institution Name]
[University Name]

Academic Year [Academic Year]
[Place]

Abstract

NetSentinel AI Lab is a local research application for reproducible study of network behaviour, anomaly detection and model explanation. It addresses the difficulty of demonstrating machine learning on private, unpredictable live traffic by generating virtual packet metadata for routine, bulk-transfer, fanout, SYN-burst and UDP-burst scenarios. The simulator emits no generated network traffic. It reuses the original sensor aggregation contract to produce seven features over ten-second windows, while preserving incomplete observations and explicit SIMULATION provenance.

The system combines a Next.js and React workbench, an authenticated Django job API, a separate SQLite demonstration database and an independent Python worker. Complete runs are assigned to training, validation and test partitions. Isolation Forest is trained on benign windows and calibrated on benign validation scores. Random Forest predicts generated behaviour families and is compared with a most-frequent baseline and fixed heuristic rules. Selected classifier predictions are explained using SHAP with an independent numerical additivity check. Cancellation, bounded history, artifact hashes, restart persistence and semantic PDF export support an inspectable experiment lifecycle.

In the retained default seed-42 experiment, 1500 windows and 178812 metadata events produced 293 complete held-out test observations. Random Forest achieved 88.74 percent accuracy and macro F1 of 0.8774, compared with 0.7862 for heuristic rules and 0.1077 for the dummy baseline. The anomaly detector achieved 87.50 percent precision but only 5.51 percent challenge recall at its conservative threshold. A three-seed study produced mean Random Forest macro F1 of 0.8806. These results demonstrate performance within the generator and do not establish real-world intrusion detection accuracy.

The project delivers a usable and tested academic prototype with transparent limitations. Its contribution is the integrated reproducible workflow, scientific separation and explanation validation rather than a new learning algorithm. The existing public connection helper, Windows companion preview and research monitor are retained with separate security boundaries. External dataset validation, current PostgreSQL acceptance and production Windows release remain distinct future gates.

Keywords: network anomaly detection; reproducible simulation; Isolation Forest; Random Forest; SHAP; explainable machine learning; provenance.

Table of Contents

Abstract.....	2
Chapter 1 Introduction.....	9
1 1 Background.....	9
1 2 Problem statement	9
1 3 Aim and objectives	9
1 4 Research questions	10
1 5 Scope and contribution	10
1 6 Report organization	10
Chapter 2 System Specification.....	11
2 1 Operating environment.....	11
2 2 Software components	11
2 3 Functional requirements	12
2 4 Quality and security requirements.....	13
2 5 Data requirements and retention.....	13
Chapter 3 System Analysis.....	15
3 1 Analysis of the earlier system.....	15
3 2 Proposed system	15
3 3 Literature and component review	16
3 4 Feasibility analysis	17
3 5 Threat and failure analysis.....	17
3 6 Use cases and actors	18
Chapter 4 System Architecture and Module Description.....	19
4 1 Architectural overview	19
4 2 Scenario generation module	20
4 3 Training and evaluation module.....	20
4 4 Explanation module.....	20
4 5 Experiment service and worker	20
4 6 Browser workbench and reports	21
4 7 Supporting public utility.....	21
4 8 Supporting Windows companion and research sensor.....	22
Chapter 5 System Design	23
5 1 Design principles and notation	23
5 2 Level 1 data flow.....	23

5 3 Level 2 analysis flow.....	24
5 4 Dataset and feature design.....	26
5 5 Partition and evaluation design	27
5 6 Database architecture.....	28
5 7 Laboratory table design	30
5 8 Companion table design	31
5 9 Interface and API design	32
5 10 User interface and report design.....	33
Chapter 6 System Implementation	34
6 1 Source organization	34
6 2 Metadata generation	35
6 3 Model training	35
6 4 Evaluation calculations.....	36
6 5 SHAP implementation and compatibility handling.....	37
6 6 Durable jobs and process control	37
6 7 Authentication and validation.....	37
6 8 Supporting measurement and security implementation	38
6 9 Packaging and release state	38
Chapter 7 System Testing.....	39
7 1 Testing approach	39
7 2 Scientific correctness tests.....	40
7 3 Security and reliability tests	40
7 4 Frontend and report tests	41
7 5 Significant failures and resolutions	42
7 6 Remaining verification	42
Chapter 8 Results and Discussion	43
8 1 Experiment configuration and evidence	43
8 2 Classification results.....	43
8 3 Confusion and family level behaviour	44
8 4 Anomaly detection results	46
8 5 Repeated seeds and uncertainty	46
8 6 Explanation and performance results	47
8 7 Application results.....	48
8 8 Threats to validity and discussion	48

Chapter 9 Scope for Future Enhancement.....	50
9 1 External validation and measurement compatibility	50
9 2 Scientific extensions	50
9 3 Usability and assistant features	50
9 4 Windows and release engineering	50
Chapter 10 Conclusion	52
10 1 Project outcome	52
10 2 Achievement of objectives	52
10 3 Concluding remarks.....	52
Bibliography	53
Appendix A Installation and User Manual	55
A 1 Laboratory installation.....	55
A 2 Running an experiment.....	55
A 3 Shutdown and recovery	55
A 4 Independent command line use	56
A 5 Supporting applications	56
Appendix B API and Data Contracts.....	57
B 1 Configuration contract	57
B 2 Job requests and responses	57
B 3 Result schema	58
B 4 Boundary failures.....	58
Appendix C Database Data Dictionary	59
C 1 Dictionary conventions	59
C MonitoringSession fields	59
C CaptureStatus fields	60
C TelemetrySample fields	60
C TrafficWindow fields.....	62
C ModelVersion fields	63
C AnomalyResult fields	64
C QueueGuard fields	64
C Companion apps fields.....	65
C Companion usage fields.....	65
C Companion policies fields	66
C Companion events fields.....	66

C Companion flows fields.....	67
C Companion destinations fields.....	67
C Companion settings fields.....	67
C 2 Integrity and normalization.....	68
C 3 Migration and storage procedures.....	68
Appendix D Test Execution and Acceptance Procedures	69
D 1 Repeatable commands	69
D 2 Manual academic demonstration	69
D 3 Remaining Windows verification	70
Appendix E Evidence Register and Release Handoff	71
E 1 Source and generated evidence	71
E 2 Release classification	71
E 3 License and attribution record.....	72
Appendix F Glossary and Viva Notes	73
F 1 Glossary.....	73
F 2 Questions for the project demonstration	73
F 3 Submission completion fields	74

List of Figures

Figure 1 Overall architecture and public local trust boundaries.....	19
Figure 2 Experiment job states and interruption paths	21
Figure 3 Context DFD for the AI laboratory	23
Figure 4 Level 1 DFD for experiment processing.....	24
Figure 5 Level 2 DFD decomposition of model analysis.....	25
Figure 6 Analysis workflow from configuration to report	26
Figure 7 Default run partitions and their permitted uses	28
Figure 8 Research entity relationships and separate laboratory tables	30
Figure 9 Companion entities and enforced versus logical references	32
Figure 10 Measured macro F1 comparison on the seed 42 test set	44
Figure 11 Random Forest confusion matrix with actual families as rows	45
Figure 12 Implemented desktop workbench with actual simulation results	48

List of Tables

Table 1	Environment and operating requirements	11
Table 2	Principal pinned software components.....	12
Table 3	Requirements and implementation acceptance	13
Table 4	Implemented retention and execution bounds.....	14
Table 5	Comparison of project approaches	16
Table 6	Principal risks and implemented responses.....	18
Table 7	Frozen host v1 feature definitions	27
Table 8	Actual application database inventory	29
Table 9	ExperimentJob field design	31
Table 10	Laboratory API summary	33
Table 11	Implementation modules and responsibilities	34
Table 12	Models and fitting protocol	36
Table 13	Recorded application verification ledger	39
Table 14	Scientific test cases and acceptance intent	40
Table 15	Integration and boundary test cases.....	41
Table 16	Default experiment dataset and split counts.....	43
Table 17	Default held out classification metrics	44
Table 18	Random Forest per class held out performance	45
Table 19	Default Isolation Forest operating point.....	46
Table 20	Repeated seed classification and anomaly results	47
Table 21	Measured single experiment performance	47
Table 22	Prioritized remaining work.....	51
Table 23	Accepted scenario configuration fields	57
Table 24	Principal lab result v1 sections.....	58
Table 25	monitoring MonitoringSession data dictionary	59
Table 26	monitoring CaptureStatus data dictionary	60
Table 27	telemetry TelemetrySample data dictionary.....	60
Table 28	telemetry TrafficWindow data dictionary	62
Table 29	detection ModelVersion data dictionary	63
Table 30	detection AnomalyResult data dictionary	64
Table 31	experiments QueueGuard data dictionary	64
Table 32	Companion apps physical schema.....	65
Table 33	Companion usage physical schema.....	65
Table 34	Companion policies physical schema.....	66
Table 35	Companion events physical schema.....	66
Table 36	Companion flows physical schema	67
Table 37	Companion destinations physical schema	67
Table 38	Companion settings physical schema.....	67
Table 39	Retained evidence locations relative to the repository	71
Table 40	Project abbreviations and terms	73

Chapter 1 Introduction

1 1 Background

Network analysis must identify the source and completeness of an observation, its calculation and its limitations. A large transfer may be legitimate, and many destinations may reflect ordinary application behaviour. An unusual measurement therefore requires interpretation rather than an automatic declaration of an attack.

Repeatability is another difficulty. A live demonstration may produce few useful examples, expose private information or be difficult to reproduce. Disruptive traffic also creates unnecessary operational risk. NetSentinel addresses these problems with a local laboratory that generates virtual packet metadata, reuses a shared feature extractor, trains real models and evaluates separate generated runs.

The project retains a public connection helper, Windows companion preview and original local research monitor. Their observation and privilege boundaries differ. The AI laboratory is the central subject of this report, with supporting modules documented for maintenance of the complete repository [20].

1 2 Problem statement

Students and junior analysts need a controlled environment in which they can compare ordinary and challenging network patterns, observe statistical errors and explain model decisions. A conventional dashboard alone cannot demonstrate training methodology, data separation or model limitations. Conversely, an isolated notebook often provides weak application integration, incomplete operational controls and no accessible demonstration workflow.

The project problem is to implement a reproducible and explainable network behaviour research prototype that connects scenario design, bounded execution, genuine machine learning, retained evidence and usable reports while preserving privacy and distinguishing simulation from live measurement. The problem includes reliable cancellation, authenticated local access, restart persistence and an honest presentation of missing observations.

1 3 Aim and objectives

The aim is to design and evaluate a local application for explainable network anomaly research using reproducible metadata simulation. The implementation supports the following objectives.

1. Generate seeded routine, bulk transfer, fanout, SYN burst and UDP burst scenarios without transmitting generated packets.
2. Extract a fixed seven-feature representation through the existing sensor aggregation code and preserve incomplete observations explicitly.
3. Train Isolation Forest for deviation detection and Random Forest for scenario classification, with simple comparison baselines.
4. Separate whole runs into training, validation and test sets, and calculate metrics from held-out observations.

5. Explain selected classifier outputs through checked SHAP contributions and training-derived reference values.
6. Provide an authenticated local workbench with actual progress, cancellation, playback, experiment history and semantic reports.
7. Preserve the separate public utility and optional Windows components without exposing local privileged controls to the internet.

1 4 Research questions

RQ1 asks whether a learned classifier distinguishes the generated scenario families more effectively than fixed rules and a most-frequent baseline on unseen runs from the same generator. RQ2 asks how a benign-trained anomaly detector behaves when its threshold is calibrated using a conservative benign validation percentile. RQ3 asks whether the explanation implementation reproduces the classifier output within a stated tolerance. RQ4 asks whether the complete local workflow remains usable and bounded when a job is cancelled, interrupted or reopened after restart.

These questions deliberately avoid a claim that synthetic performance establishes real intrusion detection accuracy. The experiment can test reproducibility, arithmetic, separation and behaviour within its generator. It cannot determine the prevalence of attacks on a real network or prove that a computer is safe.

1 5 Scope and contribution

The principal contribution is the integration of a reproducible experiment protocol with a functioning application: shared feature semantics, controlled jobs, provenance-aware outputs, checked explanations and downloadable evidence. Isolation Forest, Random Forest and SHAP are established open-source methods credited to their authors [1–5]. NetSentinel does not claim to invent those algorithms.

The implemented scope includes the independent command-line laboratory, the local browser workbench and a bounded repeated-seed study. The public website remains a connection utility. A public AI demonstration, a local language-model assistant, external dataset evaluation, production signing and renewed live hardware acceptance are future work. No synthetic model is automatically promoted to LIVE, and no anomaly score triggers a firewall block.

1 6 Report organization

The report proceeds through specification, analysis, architecture, design, implementation, testing and measured results before discussing future work and conclusions. Appendices provide setup, API contracts, database fields, evidence and demonstration procedures.

Chapter 2 System Specification

2 1 Operating environment

The reference environment is a Windows x64 student laptop. The completed AI demonstration uses Python 3.11, separate laboratory and backend environments, and a Node.js frontend. A GPU, capture driver, administrator account and external paid API are not needed for the simulation workflow. Model fitting runs in a local worker process rather than in a browser or Django request.

The values in Table 1 distinguish software requirements from planning recommendations. An 8 GB RAM machine and several gigabytes of free storage are reasonable setup recommendations for development, but they are not experimentally established minimum specifications. The observed process memory in Chapter 8 concerns one bounded run and does not include every process or the operating system.

Table 1 *Environment and operating requirements*

Area	Specification	Scope
Operating system	Windows x64	Reference platform for the local demonstration
Python	3.11 with separate environments	Laboratory and backend dependencies
Node.js	22 or later	Frontend build and local server
Memory	8 GB recommended for development	Planning guidance, not a measured minimum
Storage	Several GB of free space recommended	Source, environments, builds and bounded artifacts
Browser	Modern JavaScript browser	Desktop and 360 px layout verification recorded
Network	Not required by core lab CLI	Initial downloads and public external tests require access
Privileges	Standard user for AI Lab	Native capture and optional broker have separate prerequisites

2 2 Software components

The project preserves its existing frontend lockfile and backend requirements and uses a separate hash-pinned laboratory lock. This separation avoids changing the original live model environment solely to support simulation explanations. The standalone demonstration selects

an explicit SQLite configuration. The original research configuration continues to require PostgreSQL; it does not silently fall back to SQLite.

Table 2 *Principal pinned software components*

Component	Version	Role
Python	3.11	Laboratory and native package runtime
Django	5.2.17	Backend and job persistence
Django REST Framework	3.17.2	REST interface
Next.js	16.3.8	Frontend and local gateway
React	19.3.0	User interface
TypeScript	5.9.3	Frontend type checking
scikit-learn	1.9.0	Estimators and scientific utilities
SHAP	0.51.0	Checked feature contributions
jsPDF	4.2.1	Browser PDF creation
AutoTable	5.0.8	Semantic PDF tables
Cloudflare speedtest	1.14.1	Optional public throughput measurement

2.3 Functional requirements

The scenario form accepts a seed, runs per family, windows per run, intensity, noise and missing-observation probability. The server validates the complete configuration and rejects unknown fields and invalid numeric values. A user can start a job, observe its actual stage, request cancellation, reopen retained results, play through computed test windows and export a report. Empty and unavailable states must remain visible rather than being filled with demonstration values.

The scientific functions include dataset hashing, complete-run partitioning, train-only reference statistics, classifier and anomaly metrics, a confusion matrix, explanation additivity checks and a reproducible saved model bundle. The comparison study adds repeated seeds, whole-run bootstrap intervals and an excluded-family experiment. The supporting Windows companion discovers observed executables, estimates attributed usage, manages quota intent and presents available Defender and firewall information.

Table 3 Requirements and implementation acceptance

ID	Requirement	Evidence or boundary
FR01	Repeatable scenario generation	Same configuration reproduces dataset content
FR02	Real fitting and held-out evaluation	IF, RF, rules and dummy results calculated
FR03	Explain selected outputs	Independent 0.00001 additivity check
FR04	Start and cancel bounded jobs	Authenticated queue and worker lifecycle
FR05	Retain and reopen results	SQLite restart persistence demonstrated
FR06	Export useful evidence	Semantic PDF and JSON from completed results
FR07	Preserve provenance	SIMULATION fixed in lab contracts
FR08	Maintain supporting applications	Public, research and companion boundaries retained
NFR01	Protect local actions	Session, origin, CSRF and scoped API checks
NFR02	Remain usable on a laptop	Bounded pipeline and measured process resources

2 4 Quality and security requirements

Correctness requires preserving feature units, numerical bounds and mode labels from generation through reporting. Reliability requires bounded jobs, explicit failures, retained terminal state and cleanup of owned worker processes. Reproducibility requires the configuration, random seed, dataset digest, split manifest, model parameters and environment evidence. Usability requires understandable labels, visible limitations, keyboard access and responsive layouts.

Sensitive operations require authorization at the server boundary. The browser never receives a worker lease or backend credential. The job API accepts a fixed configuration schema rather than uploaded scripts, commands, model files or target URLs. The standalone servers bind to loopback and run without elevation. Only the separately launched optional native broker may perform its narrow Windows operations.

2 5 Data requirements and retention

Laboratory records contain generated metadata and calculated results. The simulation uses documentation address ranges and does not collect payloads, cookies, passwords, browsing URLs or personal messages [16]. Job rows and owned artifact directories are bounded. The

model bundle is a private local artifact; a digest supports integrity checking but does not make an unknown serialized model trustworthy.

Table 4 *Implemented retention and execution bounds*

Area	Limit	Qualification
Experiment execution	20000 windows; 2000000 events	Over-budget work fails explicitly
Job scheduling	4 queued and 1 running	Active work is not evicted
Job history	20 retained jobs	Old terminal rows pruned during operations
Worker lease and timeout	30 s lease; 600 s job timeout	Expiry processed on next queue access
HTTP check history	10 checks for 7 days	Optional browser storage
Speed history	30 completed tests for 30 days	Separate optional store; no provider inventory
Companion discovery	512 executable identities	Observed paths only
Companion usage	90 days	UTC daily buckets, separated by mode
Companion flows	10000 rows and 7 days	Metadata observations, not full connections
Companion events	2000 rows and 90 days	Bounded control and alert history
Destinations	5000 records and 30 days	Local observation history

The public HTTP history and speed history are distinct stores with different limits. Provider lookup data is not automatically included in either. A companion usage total is approximate observed IP traffic; it must not be described as an internet provider billing record. A gap in sensor observation must not be converted into zero bytes or a normal model result.

Chapter 3 System Analysis

3 1 Analysis of the earlier system

The earlier research system had an independent Windows sensor, Django persistence, an Isolation Forest workflow and a local dashboard. These are valuable foundations because capture does not depend on the web server, and the host-v1 feature representation already defines consistent aggregation semantics. The public helper subsequently added connection checks, guided troubleshooting, throughput measurement and reports. A native companion added application accounting and Windows security integration.

However, these features did not by themselves form a coherent AI research demonstration. Live traffic could be quiet or private; the application had no integrated scenario-to-training workflow; comparison baselines and explanation compatibility needed explicit evidence. Historical documentation also referred to several different project stages. The current implementation therefore adds a separate simulation laboratory while retaining existing modules and dated evidence.

3 2 Proposed system

The proposed and implemented system allows the user to design an experiment, generate metadata, train models, inspect results and save evidence in one local workflow. The generated dataset is deterministic for a fixed configuration and implementation version. Whole runs are assigned to partitions before windows are generated. Predictor input consists of numeric features; truth labels and identifiers remain separate.

The workbench is backed by a durable job API and one independent worker. This permits cancellation and explicit failure handling without holding an HTTP request open for model fitting. A local SQLite mode makes the AI demonstration usable when the original PostgreSQL service is unavailable. This is a deliberate second runtime, not evidence that the PostgreSQL deployment has passed its current acceptance gate.

Table 5 Comparison of project approaches

Criterion	Earlier workflow	Implemented AI Lab
Input availability	Depends on observed host activity	Seeded metadata scenarios
Scientific repeatability	Requires retained suitable captures	Configuration, hashes and split manifest
Model comparison	Original anomaly workflow	Classifier, anomaly model and two baselines
Explanation	Deterministic reference deviations	Checked SHAP plus training references
Demonstration	Local monitoring views	Scenario, execution, playback and report
Operational control	Independent capture and publication	Durable bounded jobs and cancellation
Interpretation	Host observation limits	Explicit synthetic scope and held-out results

3 3 Literature and component review

Isolation Forest isolates observations through random partitions, using the tendency of unusual observations to require shorter isolation paths [1]. It provides a useful unsupervised comparator when benign training data is available. The project uses the scikit-learn implementation and explicitly defines the sign and threshold of the reported deviation score [4]. It does not interpret that score as the probability of an attack.

Random Forest combines decision trees trained with randomization [2]. In this project it is used for a different question: prediction of an injected scenario family from aggregate features. The reported class probability belongs to that fitted classifier and its generated class definition. It is not a calibrated estimate of malicious intent.

SHAP associates a model output with additive feature contributions relative to a background [3]. The library offers an exact explainer suitable for the small feature space used here [5]. NetSentinel independently checks the sum against the actual classifier output. This check revealed an explanation compatibility problem and justified a disclosed fallback rather than a fabricated explanation.

CICIDS2017 is an established external intrusion-detection dataset documented by the Canadian Institute for Cybersecurity [17]. Only its official description and access information were acquired for this project. Its flow CSV data was not used for training or testing. A future adapter must establish measurement compatibility; matching column names alone is insufficient.

3 4 Feasibility analysis

Technical feasibility is supported by a completed local workflow, independently runnable pipeline, genuine model execution and automated integration evidence. The seven-feature representation permits bounded model fitting and exact explanation fallback on a laptop. A single local queue and worker are sufficient for the current use case; Redis, Celery and a distributed training platform would add complexity without addressing a measured requirement.

Operational feasibility depends on clear local startup, a password-based operator session and an accessible demonstration sequence. The source launcher creates ephemeral service credentials, starts only owned processes and preserves saved results on shutdown. The public helper remains usable without installation. Native firewall or malware scan functions remain separate explicit Windows actions.

Economic feasibility comes from reusing existing hardware and focused open-source components. The core laboratory needs no recurring cloud training or inference service. This does not imply that public bandwidth, code signing, domain registration or future deployment support are free. Those costs require a separate release decision.

3 5 Threat and failure analysis

The primary local threats are unauthorized browser requests, guessed credentials, malformed configurations, oversized results, arbitrary model deserialization and resource exhaustion. Defensive controls include exact origin validation, CSRF checks, scoped bearer credentials, bounded request bodies, generated job identifiers, independent result validation and a single running job. Forwarded headers are not treated as proof that a request originated locally.

Scientific failures are equally important. Label leakage can inflate accuracy, random window splitting can transfer near-duplicate temporal context into the test set, and presentation can hide unavailable measurements. The implemented split protocol, separate truth sidecar, train-only references and explicit unscored counts address these concerns within the generator. They do not eliminate the larger limitation that all primary experimental data comes from one simulator.

Table 6 Principal risks and implemented responses

Risk	Response	Residual limitation
Label leakage	Separate truth and numeric inputs	Shared generator assumptions remain
Temporal dependence	Partition whole runs	Few independent runs per seed
Untrusted local request	Authentication, origin and CSRF checks	Local machine compromise is outside protection
Resource exhaustion	Budgets, queue cap and timeout	Operating system load affects latency
Misleading explanation	Actual-output additivity guard	No causal guarantee
Lost observation	Null features and explicit counts	Unobserved behaviour cannot be reconstructed
Privileged misuse	Separate broker and fixed operations	Production native release gates remain
Public exposure	Static source allowlist	New publication requires artifact review

3 6 Use cases and actors

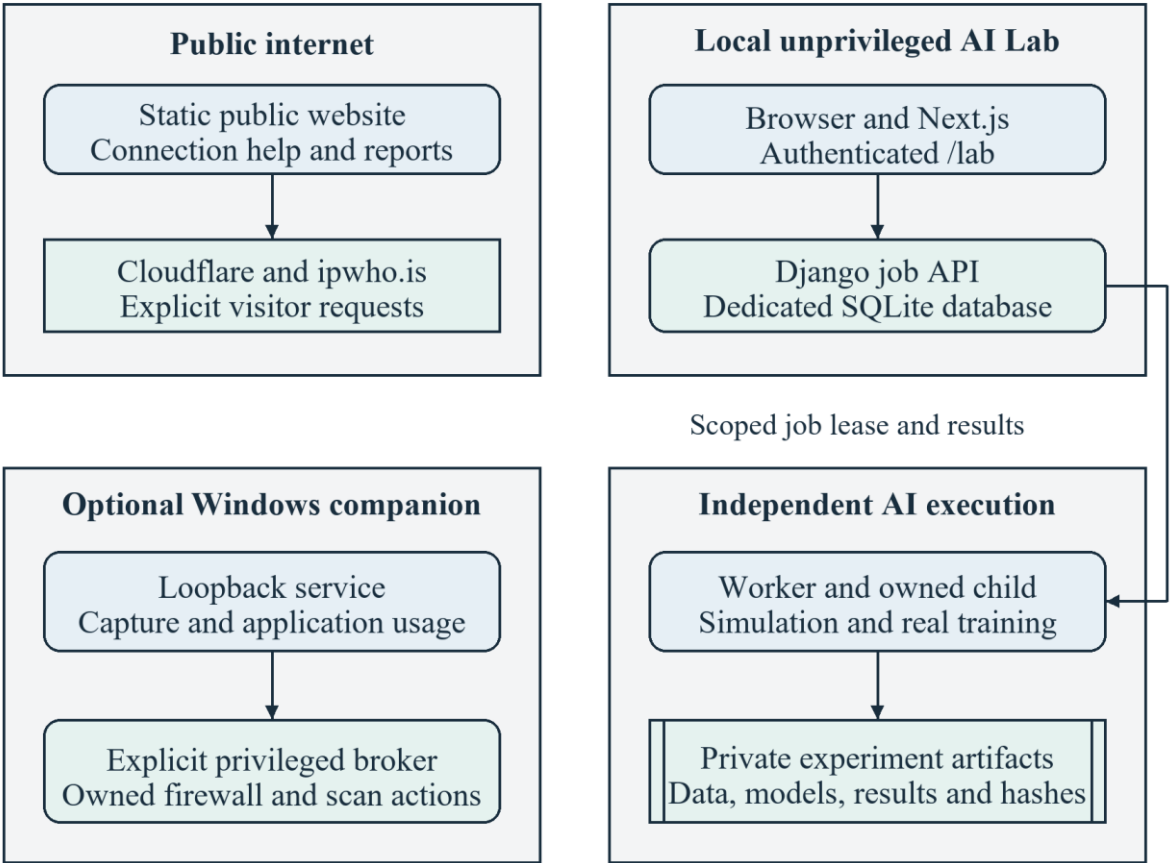
The student operator configures and runs experiments. The examiner reviews the methodology, playback, metrics and reports. A public visitor uses connection assistance in an independent browser experience. A Windows operator may explicitly use the companion and launch its broker when privileged actions are intended. These actors describe interaction responsibilities; the project does not implement a cloud organization or enterprise role hierarchy.

The principal use case begins with local sign-in, continues through scenario submission and worker completion, and ends with examination or export of a result. Alternate paths include an invalid form, a full queue, a cancelled job, an expired worker lease and a temporarily unavailable backend. Each alternate path has an explicit state; none is relabelled as a successful experiment.

Chapter 4 System Architecture and Module Description

4 1 Architectural overview

The architecture separates public browsing, local application access, independent computation and optional native privilege. The static public website cannot call a local training worker or firewall broker. The local Next.js application authenticates its operator and relays only fixed API actions to Django. Django validates and persists jobs, while a worker process generates data and fits models. The companion is a distinct local service with its own private state.



The public build has no route to the local API or broker.
Original /local research mode uses PostgreSQL separately.

Figure 1 Overall architecture and public local trust boundaries

The original research monitor is a supporting runtime using PostgreSQL, telemetry ingestion and its own model metadata. The standalone lab configuration installs only the experiment application and its routes. Therefore a successful standalone demonstration does not establish that research database migration, current live capture or privileged Windows enforcement has been verified.

4 2 Scenario generation module

The lab.simulation module constructs packet metadata for five families with deterministic per-run seeds. Routine and bulk-transfer roles represent benign workloads; fanout, SYN burst and UDP burst are challenge patterns. Noise introduces overlapping distributions, and challenge runs contain routine windows. An injected family is an experimental label, not evidence that an equivalent real pattern is malicious.

Metadata is passed through the existing sensor aggregation implementation. This avoids creating a separate feature definition solely for the classifier. The generator enforces window and event budgets and checks cancellation during work. No socket transmission, target scanning or packet payload generation is required.

4 3 Training and evaluation module

The lab.models, lab.metrics and lab.pipeline modules fit the estimators, calculate predictions and assemble the result contract. Isolation Forest uses benign training observations. Random Forest uses the complete set of valid training families. A most-frequent classifier and ordered heuristic rules provide comparison points. Calibration uses benign validation scores; the held-out test set does not determine the threshold.

The pipeline records model parameters, feature order, training window counts, reference quantiles, evaluation metrics and measured timing. lab.artifacts writes dataset, truth, split, model and environment evidence with hashes. A model can be reloaded only through the trusted local workflow and compatibility checks. Django stores a bounded result summary rather than executing serialized model files.

4 4 Explanation module

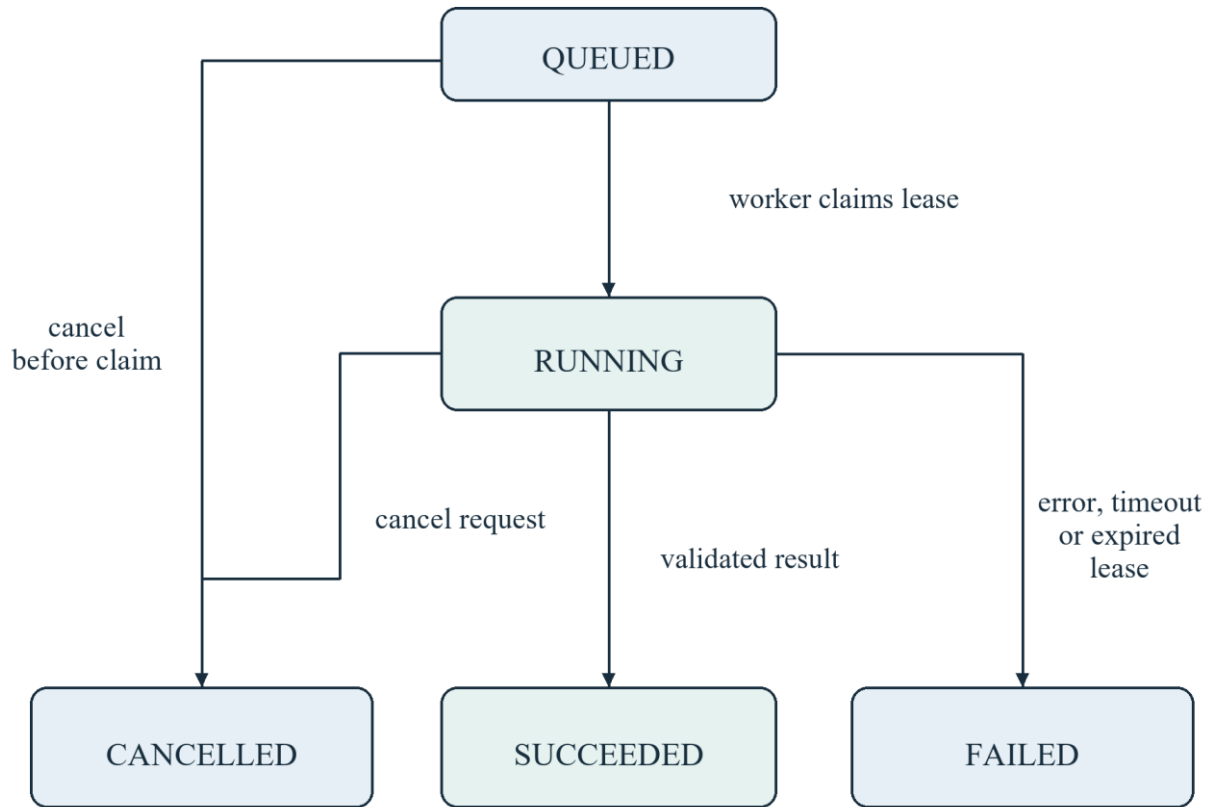
lab.explanations selects at most eight cases and uses at most 64 training rows as the SHAP background. It attempts the supported tree explanation path and independently checks all class outputs. When the contributions fail the numerical guard, a bounded ExactExplainer path evaluates the actual Random Forest prediction function with the same background. The method and reason for fallback are retained in the result.

The interface shows each feature value, a training-derived reference and its contribution for the predicted class. A positive contribution moves the output above the background expectation; a negative contribution moves it below. Contributions describe the model under its masking assumptions. They do not prove causal effects, validate the scenario label or identify an attacker.

4 5 Experiment service and worker

The experiments Django application owns admission, job transitions, leases, cancellation and retained results. Queue operations serialize access using a database-appropriate lock and an additional unique constraint for the running state. Browser read and mutation credentials are separate from the worker credential. Only the worker can claim or finish a job using an active lease.

The worker spawns an owned child for computation so that fitting can be terminated when required. It maintains a heartbeat, checks cancellation and applies a 600-second job timeout. Expired running jobs become explicit failures when the queue is next accessed. Parent-death handling and owned-process cleanup prevent an interrupted launcher from leaving an unbounded training process behind.



Terminal records are retained within the bounded history.

Figure 2 *Experiment job states and interruption paths*

4 6 Browser workbench and reports

The /lab interface comprises scenario controls, retained experiments, current job state, result summaries, recorded playback, a generated-truth overlay, model comparisons, explanations and usage guidance. Playback advances through already computed test windows. It does not generate live traffic or retrain a model for every displayed frame.

The workbench uses vanilla CSS with translucent panels and a deep teal navigation area. Compact layouts stack controls and evidence panels. Wide tables are contained in labelled scrolling regions. Error, gap, empty and stale states are treated as normal parts of the interface. Semantic PDF export uses text, tables and charts; the report is not a screenshot of the dashboard.

4 7 Supporting public utility

The public utility provides a small same-origin connection check, guided troubleshooting, a transfer-time calculator, optional history and reports. A separate explicit throughput test uses

the pinned Cloudflare engine [12]. Provider details are fetched directly from the visitor browser through ipwho.is after consent [13]. This avoids mistaking a cloud server's outbound address for the visitor's public address.

Public measurements describe their actual scope. HTTP timing includes browser and server effects; it is not ICMP ping or packet loss. Speed samples measure application payload throughput over configured requests, not maximum line capacity. A VPN or proxy may change the apparent provider and approximate location. The public export contains no private monitor, experiment API, backend credentials or model artifacts.

4 8 Supporting Windows companion and research sensor

The companion correlates observed packets with process socket snapshots, groups matching executable identities and retains ambiguous traffic as unassigned. This is approximate attribution because sockets can change between observations. Daily and monthly UTC usage buckets support quota warnings, manual intent, optional automatic blocking and an override until reset. An applied firewall rule is distinct from a measured proof that traffic stopped.

Microsoft Defender remains responsible for malware detection and remediation. NetSentinel presents documented status and explicit scan controls [10, 11]. It does not implement antivirus signatures or custom quarantine. The independent original sensor remains useful without Django, PostgreSQL or a model; its local results preserve capture quality and LIVE, SIMULATION or REPLAY provenance.

Chapter 5 System Design

5 1 Design principles and notation

The design preserves the difference between an observation, an inferred class, a statistical deviation and a control action. Each observation retains its measurement source and mode. Missing evidence remains unavailable. Authenticated control paths use fixed schemas and bounded work. Public and local builds are separated by an allowlist of source files rather than merely by hidden navigation links.

The following data flow diagrams use rectangles for external entities, rounded rectangles for processes, double-sided rectangles for data stores and directed arrows for data movement. The context diagram treats local artifact storage as an external filesystem service. Internal job and experiment stores are expanded in the level 1 diagram. A DFD shows data movement, not deployment privileges or exact execution timing.

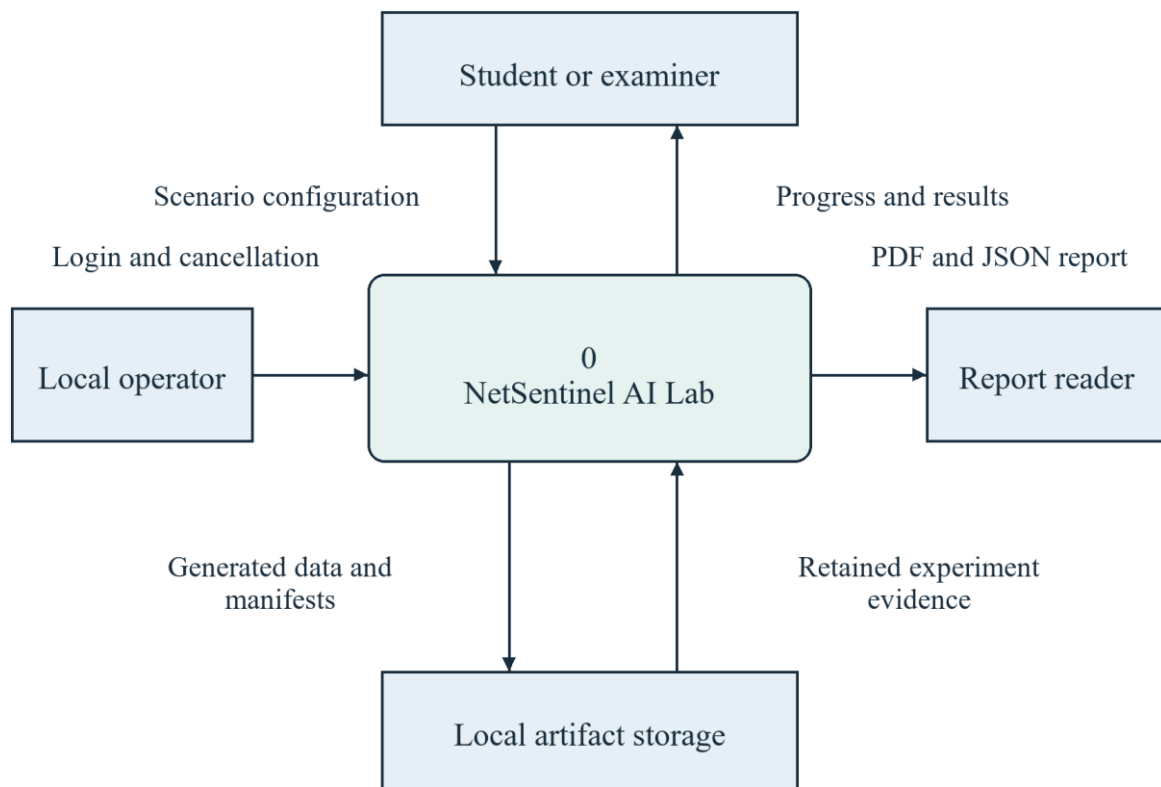


Figure 3 Context DFD for the AI laboratory

5 2 Level 1 data flow

The level 1 DFD decomposes the laboratory into validation and queuing, generation and partitioning, training and evaluation, and presentation and export. Job state belongs to the database. Numeric dataset records, truth sidecars and model files belong to the worker's private artifact area. The frontend consumes a validated result through the API rather than opening model files.

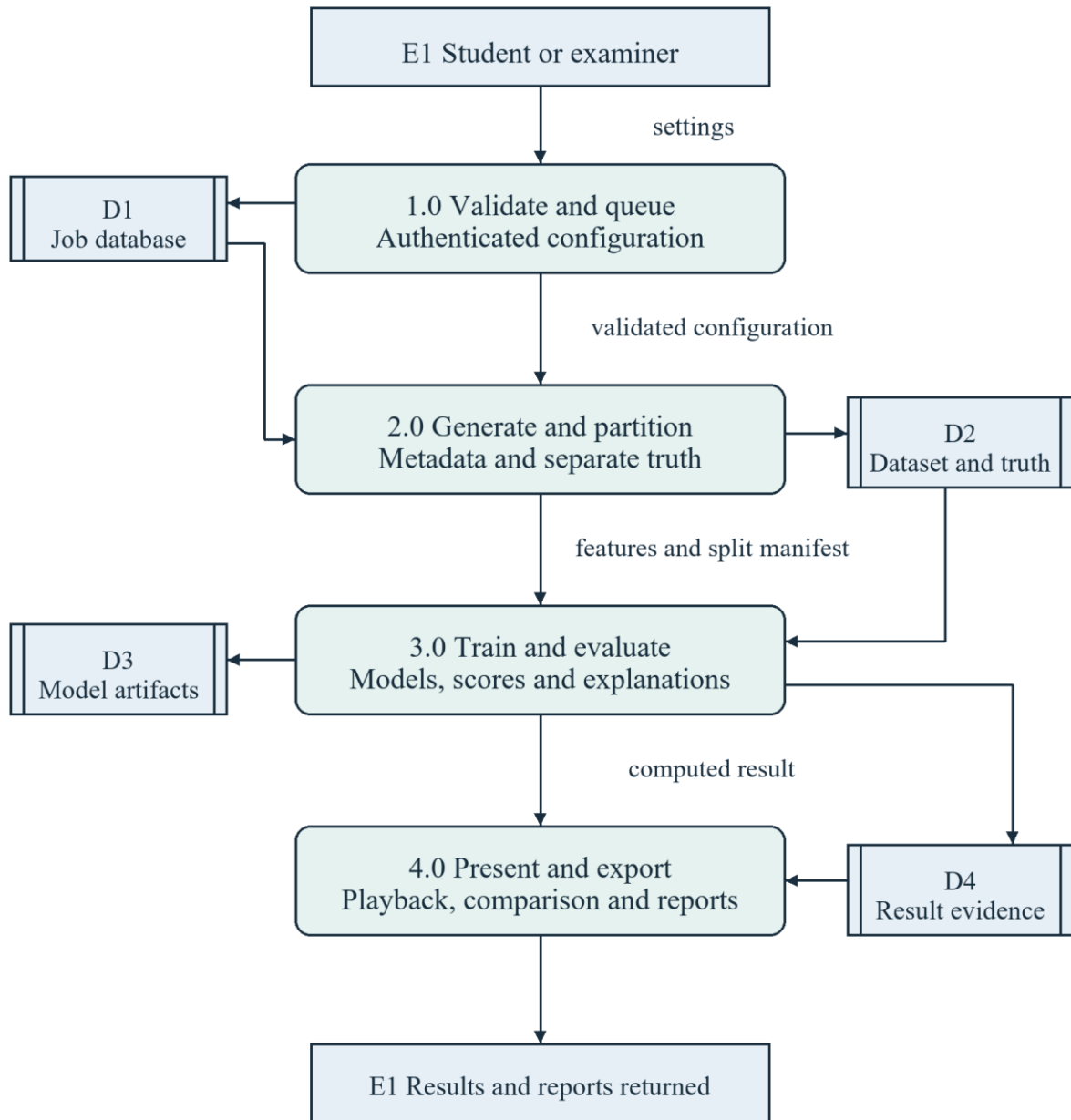


Figure 4 Level 1 DFD for experiment processing

Truth labels flow into evaluation and into the optional presentation overlay. They must not flow into the estimator's numeric input matrix. This boundary is essential because a predictor that receives its label, run identity or generation seed would produce misleading results even if the user interface and metric calculation were otherwise correct.

5 3 Level 2 analysis flow

The level 2 decomposition expands training and evaluation. The workflow first selects complete observations, then fits models, calibrates the anomaly threshold, infers on untouched test runs, compares predictions with truth and validates explanations. Results include unscored counts so that excluding incomplete records remains visible.

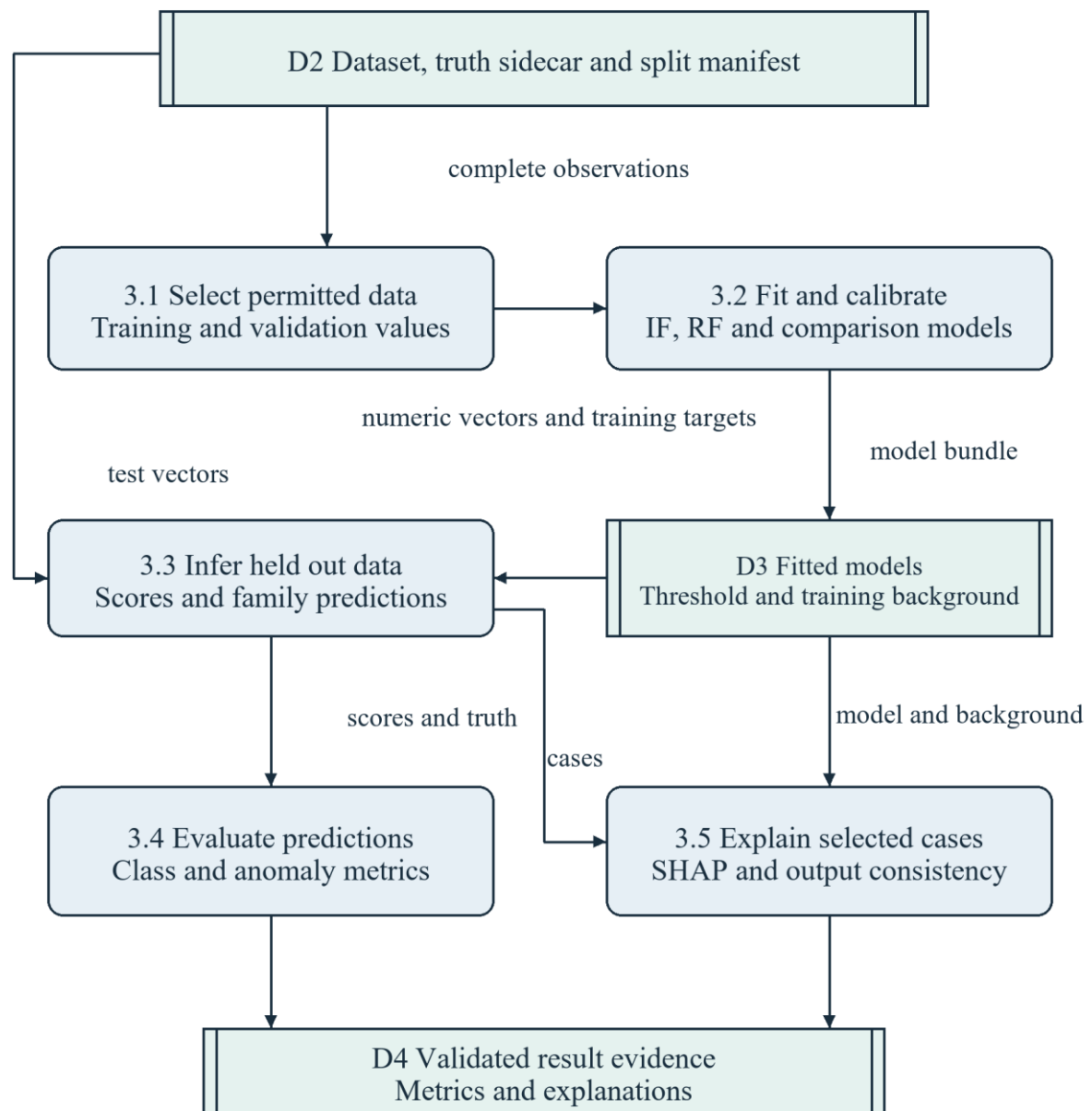


Figure 5 Level 2 DFD decomposition of model analysis

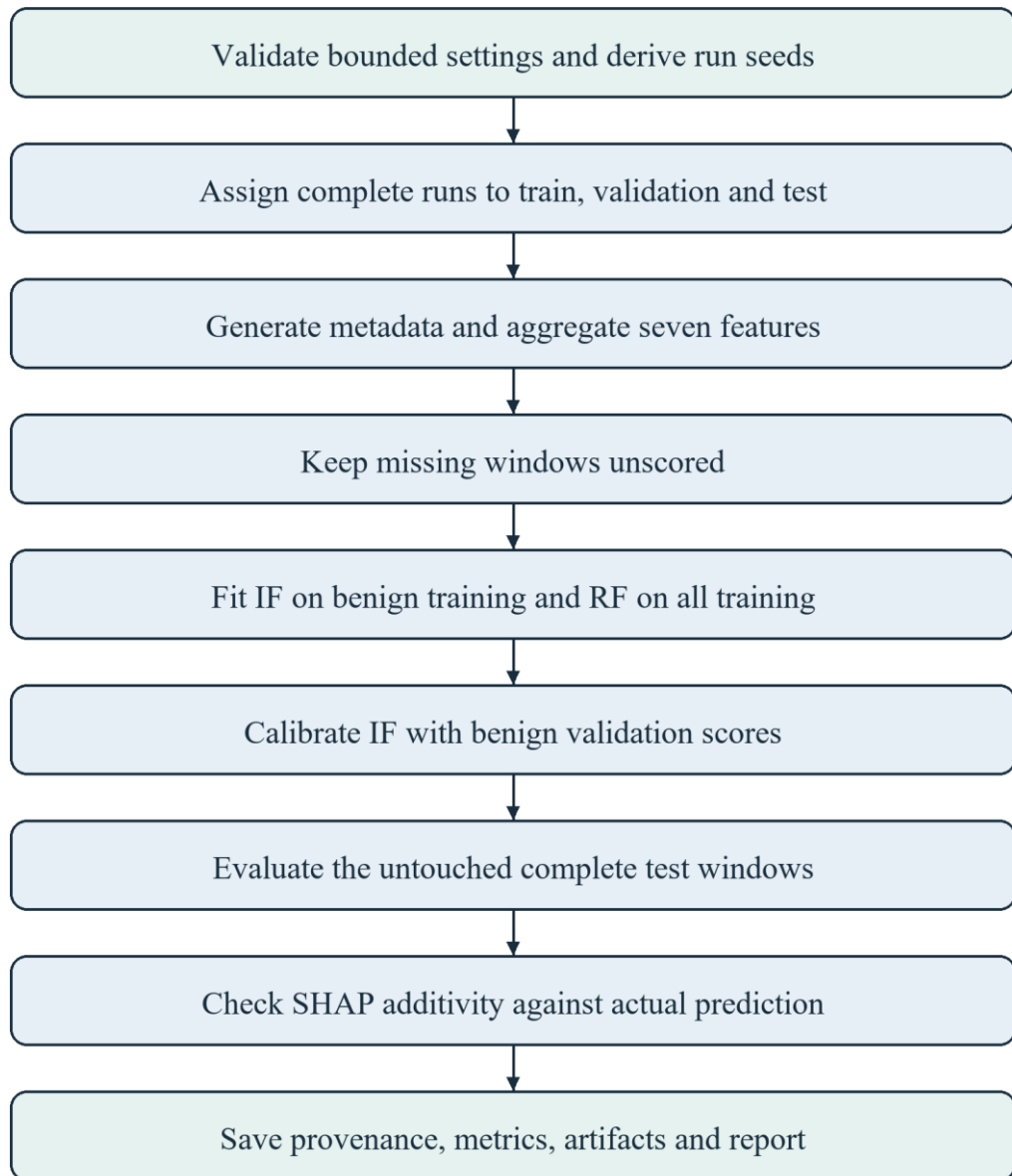


Figure 6 *Analysis workflow from configuration to report*

5 4 Dataset and feature design

The unit of model input is one complete ten-second window. The seven ordered host-v1 values are calculated from finalized directional and protocol aggregates. Ratios use a stated zero-denominator convention only when the window is otherwise complete. An invalid or partial window produces no feature vector; it is not represented by a vector of zeros.

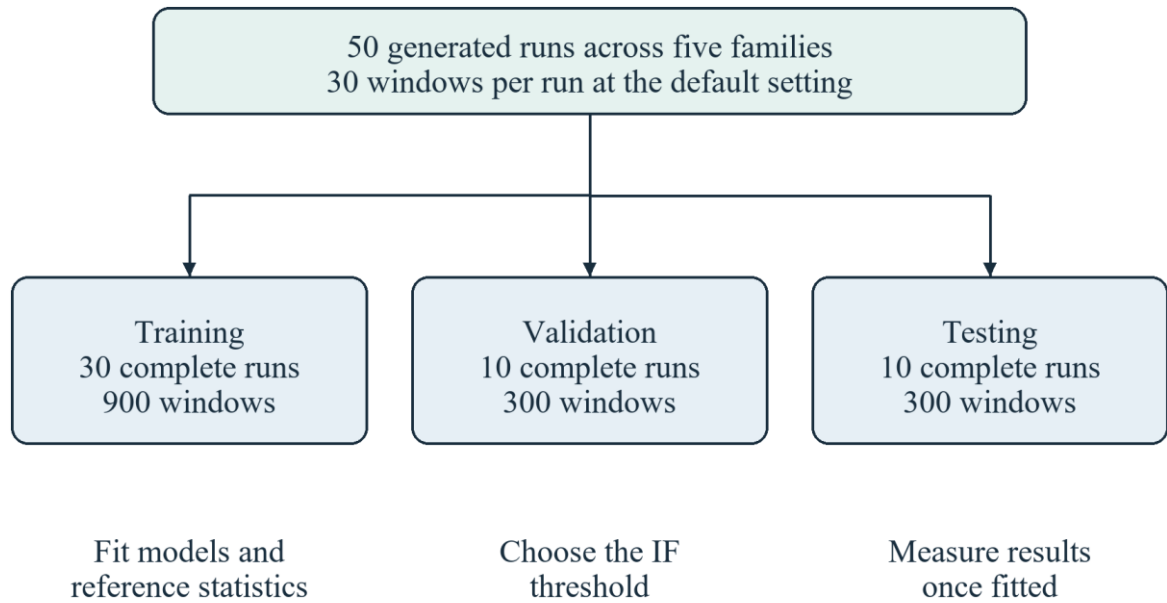
Table 7 Frozen host v1 feature definitions

Feature	Calculation	Unit
packets_per_second	Observed IP packets / 10	packets/s
ip_bytes_per_second	Observed IP bytes / 10	bytes/s
outbound_byte_fraction	Outbound bytes / known directional bytes	0 to 1
unique_remote_peers	Distinct remote IPs in complete window	count
tcp_syn_fraction	TCP SYN packets / TCP packets	0 to 1
udp_fraction	UDP packets / all observed IP packets	0 to 1
mean_ip_packet_bytes	IP bytes / observed IP packets	bytes/packet

The public speed-test units are separate from these feature units. An IP byte count in a sensor window is not a browser response-body count. Neither includes every link-layer overhead, retransmission accounting rule or internet provider charging rule. These distinctions prevent results from different products being combined as though they were the same measurement.

5 5 Partition and evaluation design

Whole runs are assigned independently within each family: floor 60 percent to training, at least one run from floor 20 percent to validation, and the remainder to testing. With the default ten runs per family, each family has six training, two validation and two test runs. There is no random shuffle of windows across these boundaries.



Incomplete windows stay in the evidence but are excluded from scoring.

Figure 7 *Default run partitions and their permitted uses*

Related windows within a generated run are not independent experimental replications. The repeated-seed study therefore resamples complete held-out runs for conditional uncertainty intervals. This preserves within-run structure. It still does not measure uncertainty over all possible training populations or establish transfer to a different traffic generator.

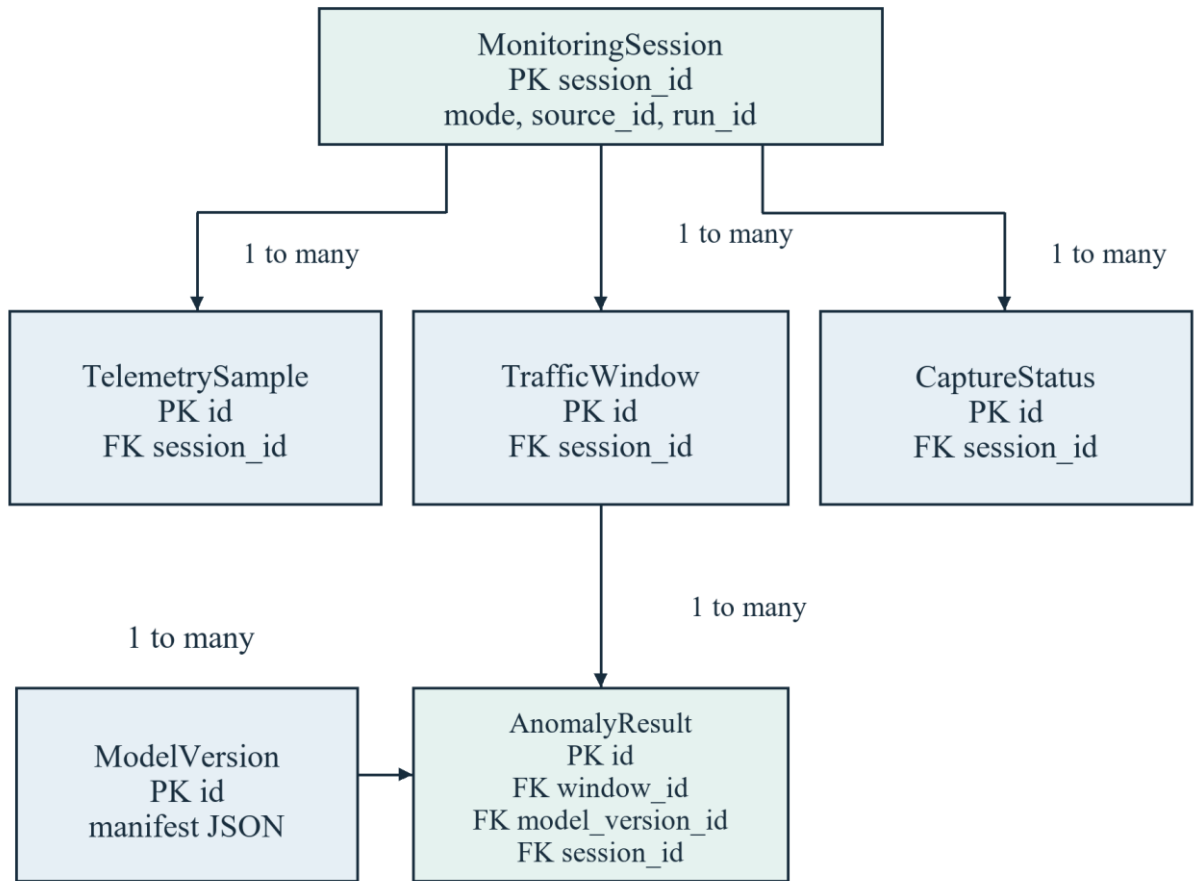
5 6 Database architecture

The repository contains three distinct persistence contexts. The original research backend uses six application models in PostgreSQL. The standalone laboratory uses ExperimentJob and QueueGuard in its own SQLite database. The companion uses seven manually defined SQLite tables. Django migration bookkeeping and framework internals are not counted as application domain tables.

Table 8 Actual application database inventory

Context	Tables	Relationship to release
Standalone lab SQLite	experiments_experimentjob; experiments_queueguard	Verified local prototype database
Original research PostgreSQL	monitoring_monitoringsession; monitoring_capturestatus; telemetry_telemetrysample; telemetry_trafficwindow; detection_modelversion; detection_anomalyresult	Source retained; current PostgreSQL acceptance blocked
Companion SQLite	apps; usage; policies; events; flows; destinations; settings	Separate native preview state

The original session model is the parent of telemetry, traffic windows and capture status. Anomaly results refer to a window, a model version and a session. Service validation verifies cross-record agreement in addition to database constraints. ModelVersion stores metadata rather than an executable model blob. The full field-level dictionary appears in Appendix C.



AnomalyResult also references its MonitoringSession.
The service validates session and window agreement.

Standalone lab tables are separate: ExperimentJob and QueueGuard.
There is no foreign key between those two tables.

Figure 8 Research entity relationships and separate laboratory tables

5 7 Laboratory table design

ExperimentJob is intentionally cohesive: its bounded configuration and result are JSON fields, while identity, state, lease and progress are typed columns with constraints. This avoids scaffolding separate dataset, model, metric and explanation tables when the application reads a completed experiment as one bounded unit. The trade-off is reduced database-level querying of individual metrics and a greater need for application schema validation.

Table 9 ExperimentJob field design

Field	Type and nullability	Purpose
id	UUID, PK	Generated job identity
status	Text 12, required	QUEUED, RUNNING, SUCCEEDED, FAILED or CANCELLED
stage	Text 48, required	Actual execution phase
created_at	UTC datetime	Creation time
updated_at	UTC datetime	Most recent saved update
config	JSON, required	Validated complete settings
cancel_requested	Boolean, default false	Cancellation intent
completed	Unsigned integer, nullable	Completed progress units
total	Unsigned integer, nullable	Expected progress units
error	Text 200, blank allowed	Fixed redacted error description
result	JSON, nullable	Validated result only on success
lease_digest	Text 64, blank allowed	Digest of active worker lease
lease_expires_at	UTC datetime, nullable	Lease renewal deadline

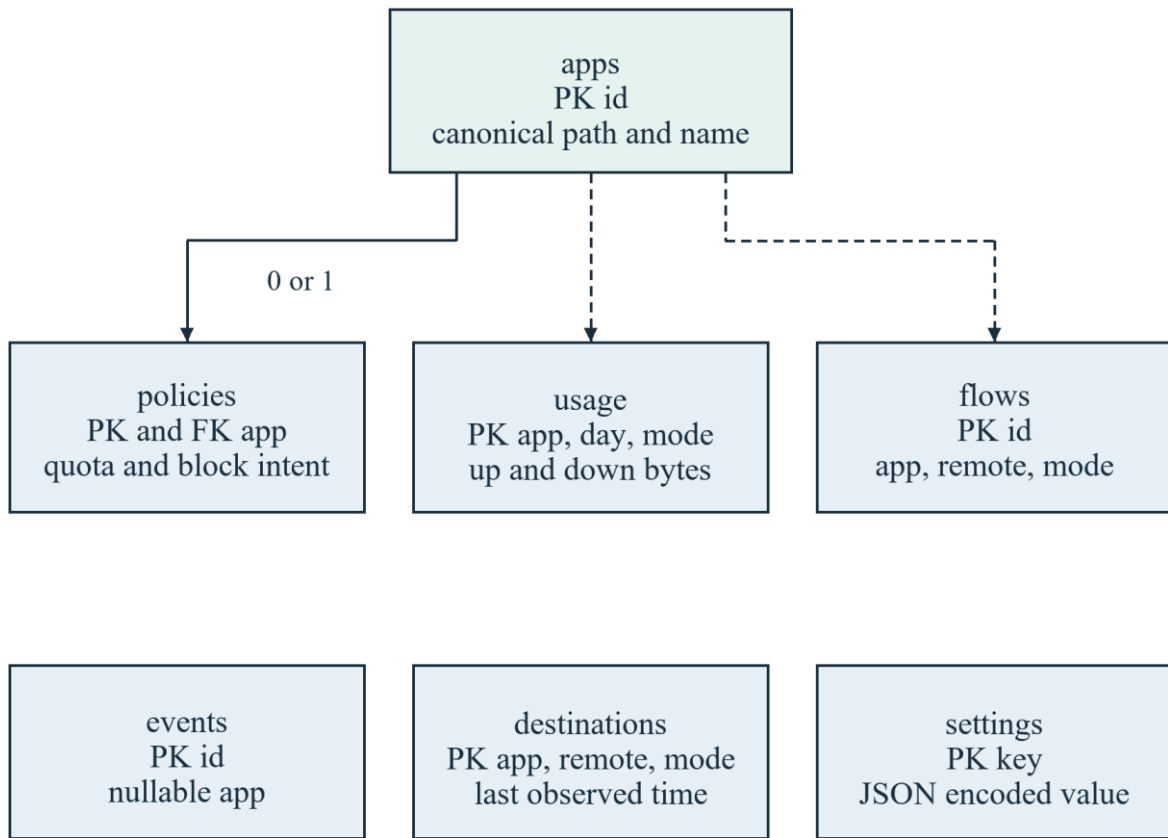
QueueGuard contains only id and revision. A check constrains id to one, and a seeded row supports the SQLite write lock before queue state is read. It has no foreign key to ExperimentJob. The PostgreSQL runtime uses an advisory transaction lock for the same admission operation. These are implementation-specific concurrency mechanisms, not interchangeable verification results.

The job constraints permit only known states, allow a non-null result only for SUCCEEDED, require completed progress not to exceed total when both are present, and allow a single RUNNING row. An index on status and created_at supports queue queries. Lease digests are never serialized to the browser. A SHA-256 digest stores the worker lease comparison value without retaining the raw lease in the row.

5.8 Companion table design

Executable identity uses a hash of a normalized canonical path so subprocesses using the same executable can be grouped. This identifies an observed executable location; it is not a cryptographic verification of the file's publisher or contents. The schema stores quota intent

separately from usage, allowing configuration to survive a restart without implying that a firewall rule is currently effective.



Solid: enforced foreign key. Dashed: logical app reference.
Events and destinations also use logical app references.

Figure 9 Companion entities and enforced versus logical references

Only policies.app declares a foreign key to apps.id in the current schema. Other app fields are logical references maintained by application logic. usage also needs the special unassigned identity. This design is documented as implemented rather than depicting stronger relational constraints that do not exist. SQLite foreign key enforcement is explicitly enabled for the connection [18].

5 9 Interface and API design

The browser creates and cancels jobs through same-origin Next.js routes. The relay checks the operator session, exact origin, CSRF token and local request marker before mutation. It forwards only fixed paths to a configured loopback backend. The backend independently authorizes the required scope and validates input. A public browser cannot obtain local authority merely by knowing an endpoint path.

Table 10 Laboratory API summary

Method and relative route	Scope	Purpose
GET jobs/ and jobs/<uuid>/	Read token	Summaries and completed result
POST jobs/	Lab token	Create a validated job
POST jobs/<uuid>/cancel/	Lab token	Request or complete cancellation
POST worker/claim/	Worker token	Claim one job with a lease
POST worker/<uuid>/heartbeat/	Worker token and lease	Renew lease and report progress
POST worker/<uuid>/finish/	Worker token and lease	Submit terminal outcome

The server returns structured availability rather than forcing every state into a percentage. Progress contains a stage and optional completed and total counts. A cancelled or failed experiment has no successful result. The worker finish endpoint receives a larger result body only after worker authentication. Appendix B defines request examples and failure handling without embedding any credentials.

5 10 User interface and report design

The navigation emphasizes tasks: configure an experiment, inspect retained work, understand results and learn how to use the application. Controls expose bounded inputs with defaults. Results distinguish classification, anomaly detection and unscored observations. The truth overlay is explicitly generated truth, and class confidence is accompanied by its model-output meaning.

The PDF design includes a report title, SIMULATION scope, generation time, configuration, model comparisons, anomaly metrics, explanation evidence and limitations. The public and companion report paths use only data available in their own environments and redact sensitive fields by default. The academic report presented here adds the source architecture, implementation detail and research analysis that are outside the shorter application-generated PDF.

Chapter 6 System Implementation

6 1 Source organization

Implementation extends the existing modular repository. The lab package contains the independent scientific workflow. The experiments Django application implements job persistence and API behaviour. The frontend lab components implement local interaction, and the launcher coordinates the processes. The existing sensor, ml and companion packages retain their distinct responsibilities.

Table 11 *Implementation modules and responsibilities*

Source area	Responsibility
lab/contracts.py	Configuration and result validation without ML imports
lab/simulation.py	Deterministic metadata and truth generation
lab/models.py and metrics.py	Fitting, prediction and evaluation arithmetic
lab/explanations.py	Checked SHAP and bounded exact fallback
lab/pipeline.py and artifacts.py	Experiment orchestration and reproducible evidence
lab/benchmark.py	Repeated seeds, group bootstrap and family exclusion
lab/worker.py	Claim, lease, owned child, cancellation and cleanup
backend/experiments/	Job model, migrations, service, validation and API
backend/config/lab_settings.py	Explicit standalone experiment-only SQLite runtime
frontend/src/components/lab/	Scenario and results workbench
frontend/src/lib/lab-*.ts	Contracts, gateway and PDF report
scripts/run_ai_lab.py	Local startup, credentials and owned-process shutdown
scripts/verify_ai_lab.py	Actual HTTP integration verification
sensor/, ml/, companion/	Preserved capture, original research ML and native preview

The separation allows the command-line experiment to run without a database or web server. Django imports lightweight validation contracts but does not import the scientific runtime or load models. The worker owns computation and its artifact directory. This boundary reduces request latency, isolates cancellation and makes the pipeline easier to test independently.

6 2 Metadata generation

The simulator validates the complete configuration before generating work. A deterministic digest-derived seed combines the experiment seed, family and run index. Each run uses its own random state, while missing-observation decisions use a separate stream. The dataset hash captures deterministic data content; repeated runs may have different wall-clock generation timestamps and execution times without changing the generated dataset.

The generator creates metadata records using reserved documentation addresses, protocol choices, directions, packet lengths and timing. It submits them to the shared WindowAggregator rather than emitting them onto a network. A generated SYN burst is therefore a pattern in stored metadata. No third-party host receives a SYN packet from the simulation.

Noise intentionally makes the classification problem imperfect. A challenge run can contain routine windows and different families can overlap in the seven-feature space. The implementation does not alter predictions to achieve a target score. Missing windows remain in the dataset evidence with null features and are excluded from training or scoring only through explicit validity selection.

6 3 Model training

Isolation Forest uses 100 estimators, a maximum sample count of 256 per tree or the available benign training count, a fixed random seed and one computational worker. The pipeline converts the library's score_samples output to a positive deviation convention by negating it. The operational threshold is the linear 99th percentile of benign validation scores, and a window is flagged only when its score is strictly greater than that threshold.

Random Forest uses 100 trees, maximum depth 12, minimum leaf size two, balanced class weights and a fixed seed. It predicts routine, bulk_transfer, fanout, syn_burst or udp_burst. Numeric vectors preserve the frozen feature order. Run identifiers, seed values, provenance labels and truth records are never columns in that vector.

The most-frequent baseline predicts the most frequent family in the training set. The heuristic comparator checks UDP fraction at least 0.7, then TCP SYN fraction at least 0.55, then at least 18 remote peers, then at least 9000 IP bytes per second; it otherwise predicts routine. These are fixed prototype heuristics, not validated network security thresholds.

Table 12 Models and fitting protocol

Model	Input partition	Fixed configuration
Isolation Forest	Complete benign training	100 trees; at most 256 samples; one worker; seeded
IF threshold	Benign validation only	Linear 99th percentile; strict greater-than test
Random Forest	All complete training families	100 trees; depth 12; minimum leaf 2; balanced weights
Most-frequent baseline	Training labels	Always predict training majority family
Heuristic rules	Test feature vectors	Fixed UDP, SYN, peer-count and byte-rate thresholds
SHAP reference	At most 64 training rows	All-family background; at most eight explained cases

6 4 Evaluation calculations

The confusion matrix uses actual labels as rows and predicted labels as columns. Accuracy is the fraction of correct predictions. Per-class precision uses the predicted class total as its denominator; recall uses the actual class support. F1 is the harmonic mean of precision and recall. Macro F1 averages the per-class F1 values, reducing the dominance of a large routine class.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

$$\text{Accuracy} = \text{Correct predictions} / \text{Scored observations}$$

$$\text{Macro F1} = \text{Sum of class F1 values} / \text{Number of classes}$$

TP denotes true positives, FP false positives and FN false negatives for the class or challenge definition being evaluated. All ratios must be interpreted with their stated unavailable or zero-denominator convention.

For classification, undefined per-class precision, recall and F1 are represented as zero under the documented metric convention. Balanced accuracy averages recall over supported classes. For anomaly evaluation, undefined ratios remain null. Average precision is available only when the evaluation includes both benign and challenge observations.

False alerts per hour divide false positives by complete benign virtual observation time, using ten seconds per scored window. This quantity is a rate within a generated experiment. It is not

a measured production alert rate. A short denominator can produce a large hourly extrapolation even when only a few windows were falsely flagged.

6 5 SHAP implementation and compatibility handling

The initial tree explanation path was checked against actual classifier probabilities rather than trusted solely because a library returned numbers. In the default experiment, the parsed model prediction matched the Random Forest, but the interventional contributions differed from the actual output by as much as 0.03018. Converting the inputs to float32 did not resolve the discrepancy.

The implementation retained the 0.00001 tolerance and introduced a bounded exact fallback using the official SHAP ExactExplainer [5]. With seven features, the coalition space contains 128 subsets. The background remains at most 64 training rows and the number of explained cases remains at most eight. Cancellation is checked between cases. The final result records the actual explanation method and limitation text.

$$\text{Model output} = \text{Base value} + \text{Sum of feature contributions}$$

This is a numerical consistency test, not a claim that the model's explanation is causally correct. Feature dependence can make independently masked combinations unrealistic. Background selection changes the reference against which a contribution is interpreted. These restrictions belong beside the reported contributions and are discussed again in Chapter 8.

6 6 Durable jobs and process control

The backend accepts up to four queued jobs and a single running job. A worker claim returns a randomly generated lease; the database stores only its digest. The worker renews a 30-second lease and sends its actual stage. On completion, the backend validates the SIMULATION result and state transition. An outstanding cancellation request takes precedence over a success attempt.

The worker runs fitting in an owned spawned child so cancellation or a timeout can stop scientific code that does not frequently return to Python callbacks. The launcher and worker terminate only processes they own and verify process identity. A server restart preserves database results but expires the browser session; an interrupted running job is later failed explicitly. Retention removes old terminal work without evicting active work.

6 7 Authentication and validation

Next.js stores the local operator session in an HttpOnly, SameSite Strict cookie. The local HTTP design is intended for loopback; it is not a ready-made internet authentication deployment. Sessions expire after four hours, the in-memory session store is capped, and failed sign-in attempts are rate limited. Backend credentials stay in the server environment and are never copied into the client bundle.

Django has distinct read, ingestion, model publication, lab mutation and lab worker scopes. The standalone launcher generates separate ephemeral credentials. Input validation rejects duplicate JSON keys, non-finite numbers, booleans used as numbers, unsupported keys, invalid UUIDs

and oversized bodies. Ordinary backend bodies are limited to 64 KiB; authenticated worker completion can use at most 4 MiB. The browser create relay has a smaller configuration body limit.

Model deserialization is not an HTTP feature. The trusted local artifact loader requires integrity and environment evidence. An integrity hash does not replace a trusted source, and the report does not recommend opening unknown joblib or pickle files. Error responses use bounded, redacted messages instead of returning arbitrary exception strings, secrets or filesystem details.

6 8 Supporting measurement and security implementation

The public speed harness starts only after a user action and consent. Its configured downloads and uploads have a nominal 15.5 MB payload budget; documented engine retries increase the maximum attempted configured payload to 62 MB, excluding protocol overhead. The harness has a nominal 45-second deadline and cancels when the page is hidden or the user stops it. Completed observations, partial observations and unavailable timing remain distinguishable.

Throughput in Mbps is calculated from valid transferred payload bytes and measured duration. Jitter is the mean absolute difference between consecutive successful latency samples and is unavailable with fewer than two samples. Download samples require compatible Resource Timing body-size evidence. These calculations do not manufacture packet loss or line capacity from a failed HTTP request.

The Windows companion uses fixed Defender and NetSecurity operations. Quota policies are based on observed usage and UTC reset boundaries. Protected system and companion executables cannot be blocked through normal application control. Cleanup targets NetSentinel-owned rules only. The privileged broker is separate from the web application, and there is no automatic block based solely on an anomaly model score.

6 9 Packaging and release state

The laboratory is delivered as source with reproducible dependency instructions and a Windows launcher. The source archive excludes private databases, credentials, generated model artifacts, packet captures and downloaded dependencies. It is unsigned; there is no claim of code-signing certification or guaranteed Windows SmartScreen acceptance. The draft pull request remains unmerged.

The public connection utility was previously published and recorded as verified on 1 October 2026. The AI laboratory has not been newly deployed to that public site. The current source branch is codex/ai-lab-prototype, with the inspected implementation and handoff at d0fbbb1. Documentation is a snapshot of that release state, not a claim that every historical product gate is currently complete.

Chapter 7 System Testing

7 1 Testing approach

Testing combines deterministic unit tests, backend integration tests, frontend tests, browser tests and actual local workflow evidence. The scientific tests verify semantics and invariants, while integration tests verify authentication, state transitions and persistence. Browser fixtures test interaction, but only a real worker experiment demonstrates that the complete path performs generation, fitting and explanation.

The verification ledger in this chapter records the latest retained runs from 5–7 October 2026. The report preparation did not repeat every application suite. Older README totals represent earlier milestones and are not combined into a new aggregate pass claim. Environment-dependent skips and the current PostgreSQL blocker remain explicit.

Table 13 *Recorded application verification ledger*

Check	Recorded outcome	Date in 2026
AI Lab tests	41 passed; 26.107 seconds	7 October
Django backend tests	70 passed; 2.504 seconds	6 October
Frontend unit tests	58 passed	7 October
Browser regression suite	31 passed; 1 opt-in LIVE skip	6 October
Independent sensor tests	93 passed	5–6 October
Original ML tests	18 passed	5–6 October
Publication boundary tests	3 passed	5–6 October
Companion suite	58 run; 53 passed; 5 environment skips	5 October
Migrations and standalone check	No drift; explicit lab check passed	6 October
Lint and type checking	Passed; final build also typechecked	6–7 October
Production frontend build	Passed after report correction	7 October
Static public export	34 files; private routes excluded	6 October
Actual HTTP integration	360-window run, auth, cancel and restart passed	7 October
Default actual browser run	1500 windows; eight checked explanations	7 October
Application PDF	10 rendered pages individually inspected	7 October

7 2 Scientific correctness tests

Generator tests check deterministic datasets, bounded work, shared aggregation parity and the absence of network transmission. Split tests confirm that whole run identities remain disjoint and that labels or identifiers do not enter predictor inputs. Incomplete observations retain their null state and do not become benign training records by default.

Model tests verify that calibration and reference statistics use permitted partitions, saved models reproduce predictions, metric denominators match hand-calculated examples and explanation sums agree with the actual model. These tests are stronger than checking whether a training command exits successfully: a pipeline can run while still leaking labels, using the wrong feature order or reporting misleading metrics.

Table 14 *Scientific test cases and acceptance intent*

ID	Condition checked	Expected outcome
SCI01	Same seed and configuration	Identical generated content digest
SCI02	Independent run partitions	No run identity crosses sets
SCI03	Truth sidecar or ID changes	Numeric predictor matrix remains independent
SCI04	Missing observation	Null feature vector and explicit unscored count
SCI05	Reference and calibration data	Training references; benign validation threshold
SCI06	Known confusion matrix	Precision, recall, F1 and support agree
SCI07	Saved trusted model	Prediction parity after reload
SCI08	SHAP returned values	All-class reconstruction within 0.00001
SCI09	Tree explanation mismatch	Disclosed exact fallback or unavailable explanation
SCI10	Budget or cancellation boundary	Explicit failure or cancellation without fake result

7 3 Security and reliability tests

Backend and gateway tests exercise missing or wrong credentials, insufficient scopes, hostile origins, CSRF failure, malformed UUIDs, duplicate keys, non-finite values and body limits. Queue tests exercise admission pressure, a single running job, concurrent claims, cancellation, stale leases, retained state and fresh-process recovery. Result validation checks nested bounds and rejects incorrect provenance.

Actual HTTP integration used isolated storage and ephemeral credentials. It exercised the Next.js to Django to worker path, completed a genuine 360-window experiment, checked SHAP, cancelled work and restarted the stack before reading retained results. This is evidence for the standalone SQLite path. It is not a substitute for PostgreSQL migration and concurrency acceptance.

Table 15 *Integration and boundary test cases*

ID	Scenario	Required behaviour
INT01	Unauthenticated local action	Request denied before mutation
INT02	Wrong scope or worker lease	Claim and finish denied
INT03	Hostile origin or missing CSRF	Browser mutation rejected
INT04	Invalid or excessive configuration	Bounded validation failure
INT05	Concurrent queue access	Four waiting maximum and one running
INT06	Cancel during execution	Owned child stops; result not marked successful
INT07	Lease expires or owner exits	Interrupted failure and bounded cleanup
INT08	Full-stack restart	Completed result remains readable after new sign-in
INT09	Public build inspected	No private APIs, credentials or model/data artifacts
INT10	Real report download	Completed file with genuine simulation evidence

7 4 Frontend and report tests

Frontend unit tests verify contracts, calculations, history validation, redaction and gateway behaviour. Browser tests cover sign-in, job interaction, result presentation, responsive layouts and report flows. A separate default 1500-window experiment was exercised through the actual browser. Desktop and 360-pixel layouts were inspected, and report downloads were found on disk.

The application-generated semantic report was rendered as ten pages and inspected individually. A pagination correction kept explanation headings with their evidence. The report uses selectable text, tables and chart elements rather than a whole-dashboard screenshot. The final production build passed after this correction, and the static public export excluded private routes and artifacts.

7 5 Significant failures and resolutions

The explanation compatibility check found a material TreeSHAP discrepancy. The resolution was to keep the numerical tolerance and use an exact evaluation of the actual classifier, with the fallback disclosed. This is a documented correction, not a reason to omit explanation checks from testing.

An early integration harness supplied only part of the required configuration and correctly received HTTP 400. The harness was corrected to send the complete contract; server validation was not weakened. A browser download event waiter timed out although the download completed. The actual file and mobile download were verified on disk. These details distinguish test harness issues from application claims.

The original PostgreSQL endpoint timed out and the expected runtime service or executables were not available. Existing data was preserved. An explicit standalone configuration was added for the AI prototype, while PostgreSQL acceptance stayed blocked. This architectural response makes the demonstration usable without misreporting the unavailable environment as tested.

7 6 Remaining verification

Production Windows acceptance still requires controlled live attribution, effective blocking checks, quota overshoot measurement, resource measurements and recovery after abnormal termination. The source preview and mocked rule tests do not establish those properties on every Windows machine. Signing and installer hardening also remain outside the verified AI prototype.

External dataset evaluation, temporal drift, feature ablation and deployment of a reviewed public AI showcase have not been completed. These are research or release extensions rather than hidden test passes. Appendix D provides a reproducible verification sequence for a future maintainer and preserves the difference between implemented and verified capabilities.

Chapter 8 Results and Discussion

8 1 Experiment configuration and evidence

The principal example is the real default browser experiment completed on 7 October 2026. It used seed 42, ten runs per family, thirty windows per run, intensity 1.0, noise 0.15 and gap probability 0.02. All input records were generated SIMULATION metadata. Its dataset digest is recorded in Appendix E so that the output can be matched to the retained artifact.

There were 1500 windows and 178812 metadata events. Of those windows, 1470 had complete feature observations and 30 remained unscored. The nominal split contained 900 training, 300 validation and 300 test windows. Test metrics used 293 complete observations, with seven test windows unavailable. These counts must accompany accuracy values because missing records are not classified as correct predictions.

Table 16 *Default experiment dataset and split counts*

Quantity	Measured value
Seed and families	42; five families
Total runs and windows	50 runs; 1500 windows
Metadata events	178812
Complete and unavailable windows	1470 complete; 30 unscored
Nominal training split	30 runs; 900 windows
Nominal validation split	10 runs; 300 windows
Nominal test split	10 runs; 300 windows
Scored test observations	293 complete; seven unscored
Simulated time per window	10 seconds

8 2 Classification results

Random Forest achieved 88.74 percent accuracy, 88.01 percent balanced accuracy and macro F1 of 0.8774 on the complete seed-42 test observations. The heuristic comparator achieved 81.23 percent accuracy and macro F1 of 0.7862. The most-frequent baseline achieved 36.86 percent accuracy but only 0.1077 macro F1. This supports the usefulness of the learned classifier for the generated scenario task.

Table 17 *Default held out classification metrics*

Model	Accuracy	Balanced accuracy	Macro F1
Random Forest	88.74%	88.01%	0.8774
Most-frequent baseline	36.86%	20.00%	0.1077
Heuristic rules	81.23%	78.55%	0.7862

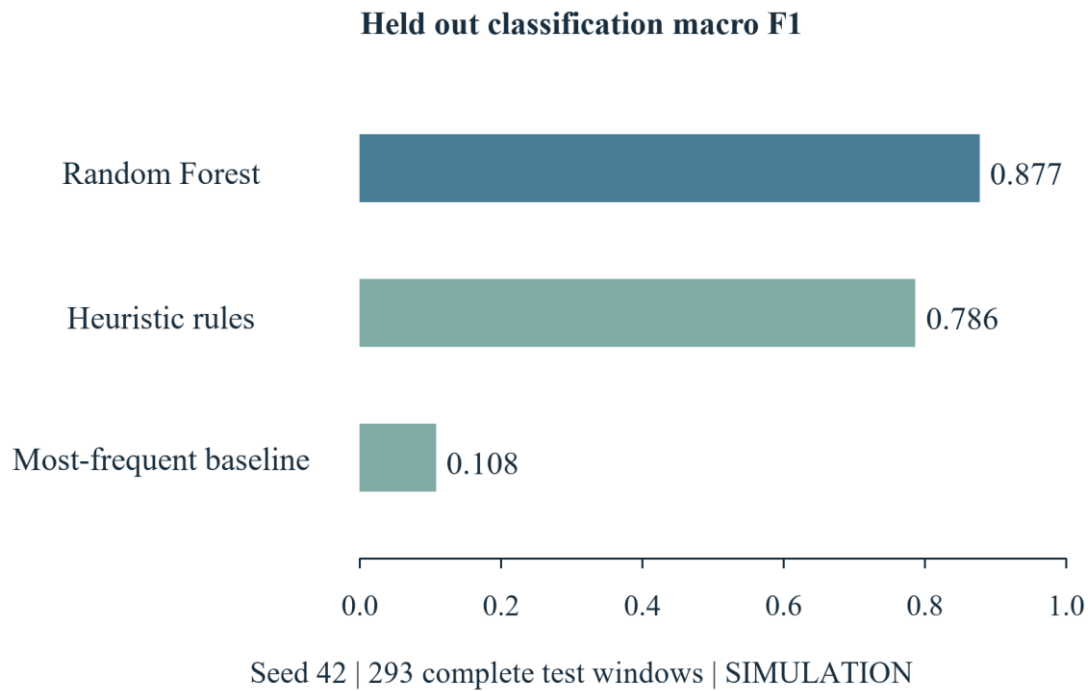


Figure 10 *Measured macro F1 comparison on the seed 42 test set*

The gap between the dummy model's accuracy and macro F1 illustrates why accuracy alone is insufficient. Routine observations make up 108 of the 293 complete test windows. Predicting routine for every observation therefore gives a non-trivial accuracy while identifying none of the other families. The balanced and macro metrics make that limitation visible.

8 3 Confusion and family level behaviour

The Random Forest correctly classified 100 routine, 47 bulk-transfer, 42 fanout, 34 SYN-burst and 37 UDP-burst observations. The confusion matrix exposes the remaining errors rather than hiding them in a single percentage. For example, four actual bulk-transfer observations were predicted as routine and four as UDP burst. These confusions are consistent with a generator that intentionally includes overlapping feature distributions.

	Predicted family				
	Routine	Bulk	Fanout	SYN	UDP
Routine	100	2	2	2	2
Bulk	4	47	1	2	4
Fanout	2	0	42	1	0
SYN	2	1	2	34	1
UDP	1	1	1	2	37

Figure 11 Random Forest confusion matrix with actual families as rows

Table 18 Random Forest per class held out performance

Family	Precision	Recall	F1	Support
routine	0.917	0.926	0.922	108
bulk transfer	0.922	0.810	0.862	58
fanout	0.875	0.933	0.903	45
syn burst	0.829	0.850	0.840	40
udp burst	0.841	0.881	0.860	42

The classifier learned relationships in the generated feature distribution. Its performance cannot establish detection of malware, successful attacks or unfamiliar real traffic. A high family probability means the fitted model favours a particular generated label among its known classes; the model has no independent observation of intent.

8 4 Anomaly detection results

The seed-42 Isolation Forest threshold was 0.650560. Precision was 0.8750, but recall was only 0.0551. The detector flagged seven of 127 challenge observations and one of 166 benign observations, leaving 120 challenges unflagged. The false-positive rate was approximately 0.60 percent, and the false-alert rate was approximately 2.17 per benign virtual hour.

Table 19 *Default Isolation Forest operating point*

Metric	Measured value	Interpretation
Threshold	0.650560	Benign validation 99th percentile
Precision	87.50%	Seven challenges among eight flags
Challenge recall	5.51%	Seven of 127 challenge windows flagged
False-positive rate	0.60%	One of 166 benign windows flagged
Average precision	0.8724	Ranking metric across score thresholds
False alerts per hour	2.1687	Per complete benign virtual hour
Unscored test windows	7	Excluded with explicit availability count

This is an important negative finding. The conservative validation threshold limited false positives but missed most challenge windows. Average precision of 0.8724 describes ranking across score thresholds; it does not repair the low recall at the selected operating threshold. A useful ranking and an unsuitable operating point can coexist.

The result answers RQ2 with a limitation, not a deployment recommendation. Any threshold revision should be specified using validation evidence and then evaluated on a fresh held-out set. Lowering the threshold after observing these test results and reporting the same set as an untouched evaluation would invalidate the protocol.

8 5 Repeated seeds and uncertainty

The repeated-seed study used seeds 42, 43 and 44 at the default configuration. Mean Random Forest macro F1 was 0.8806 with sample standard deviation 0.0104. The heuristic mean was 0.7864 with standard deviation 0.0245, and the dummy mean was 0.1059 with standard deviation 0.0020. These results indicate consistency across three generated seeds, not across real networks.

Table 20 Repeated seed classification and anomaly results

Seed	RF macro F1	IF recall	False alerts per virtual hour
42	0.8774	0.0551	2.1687
43	0.8722	0.1852	9.1139
44	0.8923	0.2901	2.2500

The per-seed Random Forest whole-run bootstrap intervals were approximately 0.557–0.903, 0.523–0.893 and 0.677–0.915. They are conditional on each fitted model. Only ten test runs are available per seed, and resampling can omit a family; the calculation retains the documented zero F1 convention for absent classes. The wide intervals discourage overinterpretation of small metric differences.

The excluded-family experiment removed complete fanout runs from training and validation, including the background windows in those runs. The classifier could not name a class absent from its training labels. Isolation Forest flagged approximately 0.0, 43.8 and 58.1 percent of the excluded-family observations across the three seeds. These variable rates are not unknown-attack detection accuracy; they are sensitivity to one withheld generated family.

8 6 Explanation and performance results

The default experiment produced eight checked exact-SHAP cases after the tree explanation path failed its guard. The fallback used the actual classifier and retained the required tolerance. RQ3 is therefore supported for the selected cases and implemented fallback, while the failed tree path remains disclosed. Additivity does not establish causal validity or explanation stability across different training backgrounds.

Table 21 Measured single experiment performance

Measurement	Observed value	Scope
Generation	3.07 s	Metadata and window creation
Training	0.30 s	Models and calibration
Evaluation	9.33 s	Metrics, selected explanations and evaluation
Total pipeline	12.71 s	One measured local run
Peak process RSS	293.2 MiB	Sampled every 25 ms
Inference p50	12.41 ms	Selected single-row inference sample
Inference p95	17.06 ms	Same timing sample, not UI latency

The measured pipeline took approximately 12.71 seconds, with a sampled peak process RSS of 293.2 MiB. Generation took 3.07 seconds, training 0.30 seconds and evaluation, including explanations, 9.33 seconds. Median single-row inference time over the selected timing sample was 12.41 ms and the 95th percentile was 17.06 ms. These are measurements from one process and run, not browser response latency or a hardware minimum.

Memory was sampled every 25 ms, so very short peaks may be missed. The process measurement includes imported libraries and does not include all browser, server and operating system memory. The repeated-seed command omits some browser-report work and should not be treated as a directly comparable end-to-end latency benchmark.

8 7 Application results

The completed workbench presents actual job progress, saved experiments, recorded test-window playback, model comparisons, explanations and PDF export. The following screenshot comes from the retained real simulation experiment rather than an illustrative metric fixture. Its SIMULATION badge is part of the product's evidence boundary.

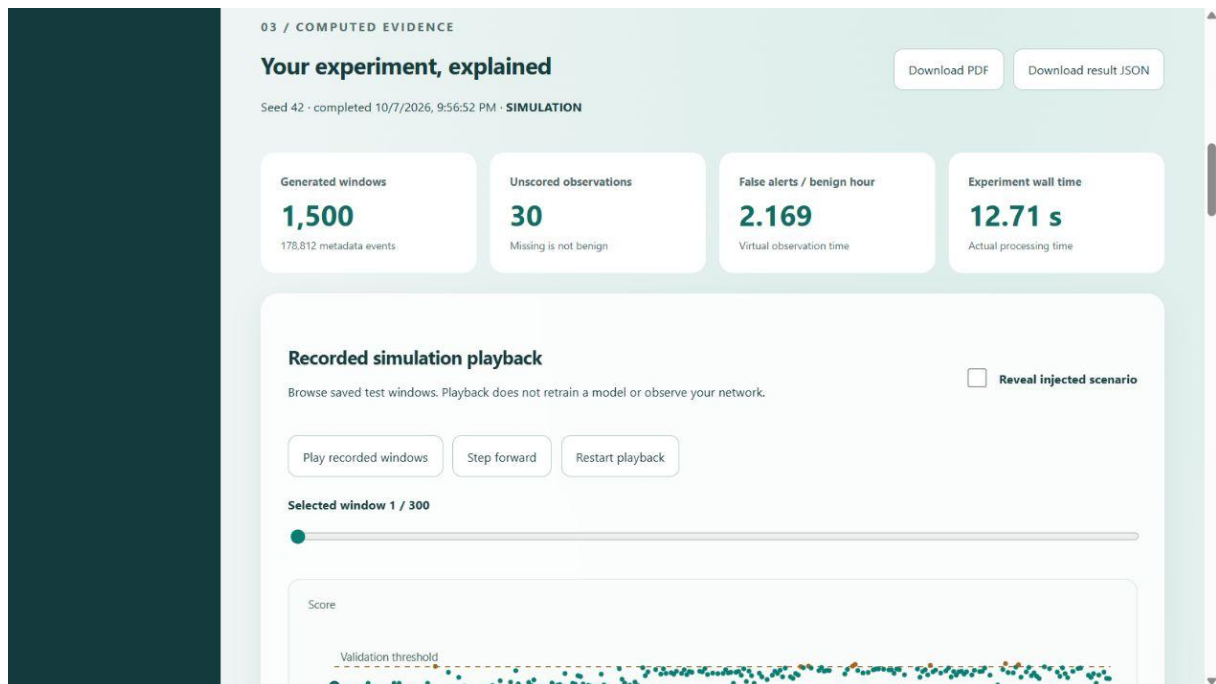


Figure 12 *Implemented desktop workbench with actual simulation results*

The independent CLI supports the same core experiment without the web application. The launcher and HTTP integration checks demonstrated cancellation and restart persistence in the standalone mode. The public build remained isolated from local routes, and no new public AI deployment was claimed. Together these results support RQ4 for the tested local workflow.

8 8 Threats to validity and discussion

Construct validity is limited by the generated labels. A fanout or burst label identifies a configured behaviour family, not malicious intent. The seven aggregate features may discard details needed to distinguish legitimate and harmful traffic. Missing observations are excluded

from metrics, so results apply to the complete observed subset rather than every nominal window.

Internal validity is improved by whole-run partitioning, separate truth records, fixed hyperparameters and train-only references. Nevertheless, train and test data share the same generator assumptions. A model may learn generator-specific regularities that do not transfer to captured traffic. Reproducible synthetic data makes such a limitation inspectable; it does not remove it.

External validity remains unestablished because there is no completed independent dataset or renewed live validation for the new lab models. The three-seed study is small. Conditional bootstrap intervals do not include every source of training uncertainty. Performance observations come from one Windows environment and bounded workload.

The project demonstrates a usable research prototype with evidence that can be inspected and reproduced. Its strongest outcomes are integrated execution, preservation of provenance and transparent model evaluation. Its weakest measured outcome is the anomaly detector's recall at the selected threshold. Reporting that weakness provides a defensible basis for future work rather than an unsupported claim of a production intrusion-prevention system.

Chapter 9 Scope for Future Enhancement

9 1 External validation and measurement compatibility

The first research extension is an independent evaluation dataset obtained through an authorized source. CICIDS2017 access remains blocked pending the official access process [17]. Before using it, an adapter must document how source fields correspond to NetSentinel's observation point, time windows, byte definitions, direction and completeness semantics. An incompatible dataset should be evaluated under a new feature contract rather than forced into host-v1.

Live validation requires a reviewed genuine baseline from the target observation profile and an ethical controlled measurement procedure. A model trained on generated scenarios must remain SIMULATION until compatibility and live validation gates are satisfied. Public-dataset accuracy alone is not sufficient for deployment on a Windows user's network.

9 2 Scientific extensions

Temporal drift experiments could train on an earlier generated or observed period and test on later conditions. Feature ablation could quantify the effect of removing peer count, byte rate or protocol fractions. A broader predeclared seed study could better characterize variance. Threshold calibration should consider a stated operating objective and use fresh test evidence after changes.

An open-set method could explicitly reject unknown behaviours rather than forcing every observation into a known family. This would require a separate definition of unknown data and appropriate evaluation. It should not be inferred from a closed-set Random Forest probability alone. Explanation stability under different background samples and dependent-feature masking also deserves investigation.

9 3 Usability and assistant features

A future local assistant could explain retained facts using a small language model, but it should receive bounded evidence and have no command execution or Windows control authority. Model weights, runtime and hardware performance have not been implemented for this feature. Any generated explanation should remain distinguishable from measured facts and deterministic calculations.

A public recorded showcase could present reviewed synthetic result assets without exposing a training API. It would need clear recorded-simulation labels, a separate export allowlist and verified hosting. User studies could measure whether students correctly interpret missing windows, probabilities and anomalies; the current report contains no invented survey results or usability scores.

9 4 Windows and release engineering

Future native release work includes a hardened installation location, protected service configuration, signed binaries, a documented update policy and measured crash recovery. Controlled live applications should verify packet attribution ambiguity, effective firewall

blocking and quota overshoot. Cleanup must continue to remove only rules owned by NetSentinel, never reset the whole firewall.

The project should remain focused. Access schedules, malicious-destination feeds, custom antivirus and speed throttling are deferred extensions. They introduce additional permissions, operational assumptions and maintenance obligations. The immediate priority is stronger scientific validation and completion of existing release gates.

Table 22 *Prioritized remaining work*

Work item	State	Acceptance requirement
PostgreSQL runtime restoration	BLOCKED	Normal-settings migration and concurrent job acceptance
Official external dataset	BLOCKED	Authorized access, license and measurement adapter
Drift and feature ablation	DEFERRED	Predeclared protocol and fresh held-out evaluation
Local language-model assistant	DEFERRED	Bounded factual context and no control authority
Public recorded AI showcase	DEFERRED	Reviewed synthetic assets and verified public deployment
Windows production release	DEFERRED	Live blocking, overshoot, recovery, hardening and signing

Chapter 10 Conclusion

10 1 Project outcome

NetSentinel implements a complete local workflow for reproducible network behaviour simulation, real model training, held-out evaluation, checked explanations and retained reports. The application connects a modular scientific pipeline to an authenticated browser workbench through a bounded job service. It preserves the independent sensor, public utility and optional Windows companion while keeping their observation and privilege boundaries separate.

The default experiment and repeated-seed study demonstrate useful classification of generated families relative to simple baselines. They also reveal a low-recall anomaly operating point and an explanation compatibility issue that required a verified fallback. These findings are part of the result. The project does not present every unusual observation as an attack or every successful status query as a security guarantee.

10 2 Achievement of objectives

The core objectives were achieved for the standalone simulation prototype: deterministic metadata generation, shared feature extraction, whole-run separation, real estimators, comparison metrics, bounded SHAP, local authentication, cancellation, persistence, playback and semantic reporting. Automated and actual integration checks support the implemented local workflow. The source, contracts, setup guide and evidence locations provide a reproducible handoff.

Remaining boundaries are explicit. PostgreSQL acceptance is blocked in the current environment. External dataset evaluation, a local language-model assistant, a public AI showcase, renewed Windows enforcement acceptance and signing are not completed. These limits do not prevent the current academic demonstration, but they prevent a claim of a production-ready intrusion prevention or antivirus product.

10 3 Concluding remarks

The project shows how an M.Sc. research application can combine scientific discipline with usable software. Reproducibility, visible missing data, protected execution and honest evaluation are as important as a model choice. NetSentinel provides a practical foundation for investigating those concerns and for extending them through independently validated data and controlled future experiments.

Bibliography

References are numbered in order of their assigned bibliography entry. Algorithm papers establish the methods; repository source and retained artifacts establish the implementation and measured results. Online references were reviewed for this report on 8 October 2026 or retained in the project resource inventory. No third-party result is presented as a NetSentinel measurement.

[1] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation Forest,” Proceedings of the Eighth IEEE International Conference on Data Mining, pp. 413–422, 2008. DOI: 10.1109/ICDM.2008.17.

<https://cs.nju.edu.cn/zhoush/zhoush.files/publication/icdm08b.pdf>

[2] L. Breiman, “Random Forests,” Machine Learning, vol. 45, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324. Author manuscript.

<https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf>

[3] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” Advances in Neural Information Processing Systems 30, 2017. arXiv:1705.07874, version 2.

<https://arxiv.org/abs/1705.07874v2>

[4] scikit-learn developers, “IsolationForest API reference.” Project implementation uses version 1.9.0; online stable documentation may advance.

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>

[5] SHAP developers, “shap.ExactExplainer API reference.” Project runtime version 0.51.0.

<https://shap.readthedocs.io/en/latest/generated/shap.ExactExplainer.html>

[6] Django Software Foundation, “Django 5.2 documentation,” models, migrations and security guidance.

<https://docs.djangoproject.com/en/5.2/>

[7] Vercel, “Next.js documentation,” application routing and deployment.

<https://nextjs.org/docs>

[8] Scapy contributors, “Scapy documentation,” packet capture and packet processing interfaces.

<https://scapy.readthedocs.io/en/latest/>

[9] psutil contributors, “psutil documentation,” process, memory and network connection APIs.

<https://psutil.readthedocs.io/en/latest/>

[10] Microsoft, “Get-MpComputerStatus,” Defender PowerShell reference.

<https://learn.microsoft.com/en-us/powershell/module/defender/get-mpcomputerstatus>

- [11] Microsoft, “New-NetFirewallRule,” NetSecurity PowerShell reference.
<https://learn.microsoft.com/en-us/powershell/module/netsecurity/new-netfirewallrule>
- [12] Cloudflare, “speedtest,” official browser speed-test component repository and measurement documentation.
<https://github.com/cloudflare/speedtest>
- [13] ipwhois.io, “IP Lookup API and IP Geolocation Documentation.” Service terms and limits must be reviewed separately before deployment.
<https://ipwhois.io/documentation>
- [14] jsPDF contributors, “jsPDF,” client-side PDF generation project.
<https://github.com/parallax/jsPDF>
- [15] ReportLab, “ReportLab PDF Toolkit User Guide.”
https://docs.reportlab.com/reportlab/userguide/ch1_intro/
- [16] J. Arkko, M. Cotton and L. Vegoda, “IPv4 Address Blocks Reserved for Documentation,” RFC 5737, January 2010.
<https://www.rfc-editor.org/rfc/rfc5737>
- [17] Canadian Institute for Cybersecurity, University of New Brunswick, “Intrusion detection evaluation dataset CIC-IDS2017.” Referenced as future evaluation data; not used in the reported experiments.
<https://www.unb.ca/cic/datasets/ids-2017.html>
- [18] SQLite authors, “SQLite Foreign Key Support.”
<https://www.sqlite.org/foreignkeys.html>
- [19] Django REST Framework contributors, “Django REST framework documentation.”
<https://www.django-rest-framework.org/>
- [20] NetSentinel project repository, branch codex/ai-lab-prototype, source checkpoint d0fbbb1. Primary implementation evidence: plan.md, docs/AI_LAB_CONTRACT.md, source models and retained experiment JSON. Reviewed 8 October 2026.
<https://github.com/hariharan4814/NetSentinel-upgraded>

Appendix A Installation and User Manual

A 1 Laboratory installation

Use Windows x64, Python 3.11 and Node.js 22 or later. Open a terminal in the repository root. The following commands describe a new source checkout; do not recreate a prepared environment unnecessarily. The laboratory lock targets Windows CPython 3.11 wheels. A different platform requires its own reviewed dependency resolution.

```
py -3.11 -m venv .venv-lab
.\.venv-lab\Scripts\python.exe -m pip install --require-hashes -r requirements-lab.lock
py -3.11 -m venv .venv-backend
.\.venv-backend\Scripts\python.exe -m pip install -r requirements-backend.txt
cd frontend
npm ci
npm run build
cd ..
.\.venv-lab\Scripts\python.exe scripts/run_ai_lab.py
```

The launcher prompts for a local password of 32 to 256 printable characters without spaces. Use the same value for the browser sign-in. It is not printed or saved by the launcher. The launcher generates distinct service credentials in memory and opens <http://127.0.0.1:3105/lab>. The backend binds to 127.0.0.1:8811. Do not launch either web server as administrator.

If the verified wheel cache is available, an offline lab installation can add `--no-index` and `--find-links resources/ai-lab/downloads/wheels` to the hash-required pip command. The resource manifest contains official origins and hashes; downloaded papers, models and datasets should not be indiscriminately copied into a public release.

A 2 Running an experiment

After sign-in, retain the default settings for a first demonstration. Start the experiment and watch the reported stage rather than expecting an invented percentage. When it succeeds, inspect the dataset counts, compare model metrics, open an explanation and use playback to inspect recorded test windows. Enable the truth overlay only when discussing the generator labels.

Repeat the same seed and configuration to compare dataset hashes. Then change one setting, such as noise, and observe the calculated result. Use cancellation on a larger bounded job to demonstrate operational control. Download the PDF or JSON from a completed result and check that its scope is SIMULATION.

A 3 Shutdown and recovery

Ctrl+C in the launcher stops its owned processes and preserves results. On the next startup, sign in with the new launcher password. If an earlier job was interrupted, the queue marks an expired running lease as failed when accessed; create a new job after resolving the cause. The application must not rewrite an interrupted job into a completed result.

The standalone state is under `artifacts/lab/standalone`. Back up or remove this directory only while its processes are stopped and only when the operator intends to remove the demonstration history. Original PostgreSQL data and companion state are separate. Occupied ports can be changed with the launcher's `frontend-port` and `backend-port` options; unrelated services must not be terminated.

A 4 Independent command line use

```
.\.venv-lab\Scripts\python.exe -m lab demo --out artifacts/lab/my-demo --seed 42
```

```
.\.venv-lab\Scripts\python.exe -m lab inspect artifacts/lab/my-demo
```

```
.\.venv-lab\Scripts\python.exe -m lab benchmark --out artifacts/lab/my-study --seeds 42 43 44
```

The command-line pipeline does not need Django, Next.js, PostgreSQL or Npcap. Use a fresh output directory. Generated model files remain trusted local artifacts and must not be downloaded from unknown sources and deserialized. Benchmark output is a separate research-study contract rather than a browser job result.

A 5 Supporting applications

The public helper requires only a browser. HTTP checks, speed testing and provider lookup are separate user actions with different data costs and external services. History is optional. The companion requires its documented Windows prerequisites and explicit capture setup; its dashboard and privileged broker are separate processes. Read `docs/COMPANION_SETUP.md` and `docs/COMPANION_UPGRADE.md` before enabling native controls.

The original /local monitor requires the original research settings and PostgreSQL. Do not serve the application with Django test settings to bypass missing production configuration. A missing Defender service, passive antivirus state, insufficient permission or unavailable capture driver must be presented as unavailable or limited rather than as a healthy security state.

Appendix B API and Data Contracts

B 1 Configuration contract

Table 23 Accepted scenario configuration fields

Field	Default	Allowed values
seed	42	Integer 0 to 2147483647
runs_per_family	10	Integer 6 to 30
windows_per_run	30	Integer 12 to 120
intensity	1.0	Finite number 0.5 to 2.0
noise	0.15	Finite number 0 to 1
gap_probability	0.02	Finite number 0 to 0.15

The mode is fixed to SIMULATION and is not a browser-supplied configuration value. Unknown keys are rejected. Integer fields reject booleans and fractional values. Float fields reject NaN and infinity. Global limits cap an experiment at 20000 windows and 2000000 metadata events, with at most 512 events in one window; reaching a limit is an explicit failure rather than silent truncation.

B 2 Job requests and responses

A create request contains one config object with every required setting. Read responses return a job identifier, state, stage, UTC timestamps, configuration, cancellation state, optional progress counts and a bounded error. A successful detail response also contains its result. Lease digests, raw credentials and worker artifact paths are omitted from browser serialization.

POST /api/v1/lab/jobs/

Content-Type: application/json

Authorization: Bearer <lab credential supplied securely>

```
{ "config": { "seed": 42, "runs_per_family": 10, "windows_per_run": 30, "intensity": 1.0, "noise": 0.15, "gap_probability": 0.02 } }
```

The worker claim response contains a job or null. A lease is returned only to the authenticated worker. Heartbeats send the active lease and actual progress. Finish accepts only SUCCEEDED, FAILED or CANCELLED, and success requires a fully validated result. A finish operation cannot be assumed replayable after a network failure; inspect retained state before deciding whether a retry is appropriate.

B 3 Result schema

Table 24 Principal lab result v1 sections

Section	Contents
schema_version and mode	lab-result-v1 and SIMULATION
generated_at and config	UTC generation timestamp and normalized settings
dataset	Digest, event/window counts, feature order and split identities
models and references	Actual parameters and training-derived quantiles
classification	Models, metrics, per-class values and confusion matrices
anomaly	Threshold, precision, recall, false positives and unavailable counts
timeline	Bounded held-out observations, predictions and generated truth
explanations	Method, model output, baseline, feature contributions and error
performance	Wall-clock stages, sampled RSS and inference sample timings
limitations	Explicit interpretation and measurement restrictions
displayed_windows and total_test_windows	Presentation count distinct from evaluation count

Numbers unavailable because evidence is missing use null, not NaN or a fabricated zero. The result exposes total and displayed window counts separately. The frontend may sample timeline points for drawing while aggregate metrics continue to use every complete test observation. Consumers must not recalculate whole-experiment accuracy from only the displayed subset.

B 4 Boundary failures

Malformed input produces a validation failure. Missing or incorrect credentials produce authentication failure, and insufficient scope denies the requested action. Queue pressure is reported explicitly, including HTTP 429 for admission when four jobs are waiting. Unknown identifiers, stale leases and invalid transitions cannot silently mutate another job. Exact error contracts are implemented in the backend and gateway test suites.

Appendix C Database Data Dictionary

C 1 Dictionary conventions

The following tables are derived from current model declarations and the companion SQL schema. Django logical field types are shown so the dictionary remains valid for the explicit laboratory SQLite mode and the original PostgreSQL mode; the physical storage representation depends on the database backend. PK means primary key, FK means declared foreign key, and nullable means that the database can store a missing value.

Inherited SessionRecord fields are repeated for clarity in each concrete research entity. It is an abstract Django class and is not an additional table. Application-level validation complements the listed column types. The tables contain metadata and aggregates; no field is intended to store packet payloads, passwords or browsing content.

C MonitoringSession fields

Table 25 monitoring MonitoringSession data dictionary

Field	Logical type	Null	Meaning
session_id	UUID	No	Primary key. Related monitoring session.
source_id	UUID	No	Source identity.
run_id	UUID	No	Groups sessions from one capture run.
interface_name	Text (255)	No	Selected local observation interface.
mode	Text (10)	No	LIVE, SIMULATION or REPLAY.
started_at	Datetime	No	Session start time.
observation_profile	Text (80)	No	Measurement scope and capability profile.
schema_version	Text (40)	No	Versioned record contract.
received_at	Datetime	No	Server receipt time.
payload_digest	Text (64)	No	Integrity and idempotency digest.

C CaptureStatus fields

Table 26 *monitoring CaptureStatus data dictionary*

Field	Logical type	Null	Meaning
id	UUID	No	Primary key. Generated or supplied row identity.
session_id	FK MonitoringSession	No	Declared foreign key; CASCADE on delete. Related monitoring session.
received_at	Datetime	No	Server receipt time.
payload_digest	Text (64)	No	Integrity and idempotency digest.
observed_at	Datetime	No	Observation time.
state	Text (20)	No	Capture lifecycle state.
valid	Boolean	No	Whether the observation is usable.
reason	Text (255)	Yes	Reason for unavailable observation.
loss_started_at	Datetime	Yes	Beginning of a monitoring gap.
gap_ended_at	Datetime	Yes	End of the monitoring gap.
monitoring_gap_seconds	Float	No	Duration of unavailable monitoring.
recovery_attempts	Unsigned integer	No	Recovery attempt count.

C TelemetrySample fields

Table 27 *telemetry TelemetrySample data dictionary*

Field	Logical type	Null	Meaning
id	UUID	No	Primary key. Generated or supplied row identity.
session_id	FK MonitoringSession	No	Declared foreign key; CASCADE on delete. Related monitoring session.
received_at	Datetime	No	Server receipt time.

Field	Logical type	Null	Meaning
payload_digest	Text (64)	No	Integrity and idempotency digest.
observed_at	Datetime	No	Observation time.
elapsed_seconds	Float	No	Actual interval used for rates.
measurement_source	Text (20)	No	Source of the measurement.
valid	Boolean	No	Whether the observation is usable.
reason	Text (255)	Yes	Reason for unavailable observation.
bytes_sent	Unsigned bigint	No	Cumulative operating-system counter.
bytes_received	Unsigned bigint	No	Cumulative operating-system counter.
packets_sent	Unsigned bigint	No	Cumulative operating-system counter.
packets_received	Unsigned bigint	No	Cumulative operating-system counter.
delta_bytes_sent	Unsigned bigint	Yes	Nullable counter difference for the interval.
delta_bytes_received	Unsigned bigint	Yes	Nullable counter difference for the interval.
delta_packets_sent	Unsigned bigint	Yes	Nullable counter difference for the interval.
delta_packets_received	Unsigned bigint	Yes	Nullable counter difference for the interval.
upload_bytes_per_second	Float	Yes	Nullable rate in bytes per second.
download_bytes_per_second	Float	Yes	Nullable rate in bytes per second.

C TrafficWindow fields

Table 28 *telemetry TrafficWindow data dictionary*

Field	Logical type	Null	Meaning
id	UUID	No	Primary key. Generated or supplied row identity.
session_id	FK MonitoringSession	No	Declared foreign key; CASCADE on delete. Related monitoring session.
received_at	Datetime	No	Server receipt time.
payload_digest	Text (64)	No	Integrity and idempotency digest.
features	JSON	Yes	Nullable seven-feature sidecar.
feature_schema_version	Text (40)	Yes	Versioned feature order and semantics.
capture_context	JSON	Yes	Observation capability context.
start	Datetime	No	Window start time.
end	Datetime	No	Window end time.
finalized_at	Datetime	No	Aggregation finalization time.
processed_at	Datetime	No	Processing time.
measurement_source	Text (20)	No	Source of the measurement.
schema_version	Text (40)	No	Versioned record contract.
partial	Boolean	No	Whether the window is incomplete.
valid	Boolean	No	Whether the observation is usable.
reason	Text (255)	Yes	Reason for unavailable observation.
packets	Unsigned bigint	No	Packets.
ip_bytes	Unsigned bigint	No	Ip bytes.
outbound_packets	Unsigned bigint	No	Outbound packets.
inbound_packets	Unsigned bigint	No	Inbound packets.

Field	Logical type	Null	Meaning
unknown_packets	Unsigned bigint	No	Unknown packets.
outbound_bytes	Unsigned bigint	No	Outbound bytes.
inbound_bytes	Unsigned bigint	No	Inbound bytes.
unknown_bytes	Unsigned bigint	No	Unknown bytes.
tcp_packets	Unsigned bigint	No	Tcp packets.
udp_packets	Unsigned bigint	No	Udp packets.
other_packets	Unsigned bigint	No	Other packets.
flow_count	Unsigned integer	No	Number of observed flow keys.
dropped	Unsigned integer	No	Dropped metadata count.
kernel_loss	Text (10)	No	Capture-loss availability state.

C ModelVersion fields

Table 29 detection ModelVersion data dictionary

Field	Logical type	Null	Meaning
id	UUID	No	Primary key. Generated or supplied row identity.
manifest	JSON	No	Versioned model metadata, not model binary.
received_at	Datetime	No	Server receipt time.
payload_digest	Text (64)	No	Integrity and idempotency digest.

C AnomalyResult fields

Table 30 *detection AnomalyResult data dictionary*

Field	Logical type	Null	Meaning
id	UUID	No	Primary key. Generated or supplied row identity.
session_id	FK MonitoringSession	No	Declared foreign key; CASCADE on delete. Related monitoring session.
received_at	Datetime	No	Server receipt time.
payload_digest	Text (64)	No	Integrity and idempotency digest.
window_id	FK telemetry. TrafficWindow	No	Declared foreign key; CASCADE on delete. Window id.
model_version_id	FK ModelVersion	No	Declared foreign key; PROTECT on delete. Model version id.
observed_at	Datetime	No	Observation time.
scored_at	Datetime	No	Analysis execution time.
anomaly_score	Float	No	Deviation score; not attack probability.
label	Text (10)	No	NORMAL or ANOMALOUS.

C QueueGuard fields

Table 31 *experiments QueueGuard data dictionary*

Field	Logical type	Null	Meaning
id	Unsigned smallint	No	Primary key. Generated or supplied row identity.
revision	Unsigned smallint	No	SQLite queue guard lock update.

C Companion apps fields

Table 32 Companion apps physical schema

Field	SQLite type	Key or null	Meaning
id	TEXT	PK; see note	Executable path hash.
path	TEXT	Required	Canonical executable path; private.
name	TEXT	Required	Executable display name.
protected	INTEGER	Required	Critical application protection flag.
seen	REAL	Required	Last observation in UTC epoch seconds.

C Companion usage fields

Table 33 Companion usage physical schema

Field	SQLite type	Key or null	Meaning
app	TEXT	PK	Logical app identity or unassigned.
day	TEXT	PK 2	UTC date bucket.
mode	TEXT	PK 3	Observation provenance.
up	INTEGER	Required	Observed outbound IP bytes.
down	INTEGER	Required	Observed inbound IP bytes.

C Companion policies fields

Table 34 *Companion policies physical schema*

Field	SQLite type	Key or null	Meaning
app	TEXT	PK and FK; see note	Primary and foreign key to apps.id.
daily	INTEGER	Nullable	Daily byte quota, nullable.
monthly	INTEGER	Nullable	Monthly byte quota, nullable.
warning	INTEGER	Required	Warning percentage, default 80.
direction	TEXT	Required	Quota direction, default combined.
auto	INTEGER	Required	Explicit automatic blocking flag.
override	REAL	Required	Temporary unblock expiry in UTC seconds.
manual	INTEGER	Required	Persisted manual block intent.

C Companion events fields

Table 35 *Companion events physical schema*

Field	SQLite type	Key or null	Meaning
id	INTEGER	PK	Row identity.
at	REAL	Required	UTC epoch timestamp.
app	TEXT	Nullable	Optional logical app identity.
category	TEXT	Required	Quota, control or security category.
code	TEXT	Required	Structured event code.
message	TEXT	Required	Bounded explanation.
mode	TEXT	Required	Observation provenance.
marker	TEXT	Nullable	Unique optional deduplication marker.

C Companion flows fields

Table 36 *Companion flows physical schema*

Field	SQLite type	Key or null	Meaning
id	INTEGER	PK	Row identity.
app	TEXT	Nullable	Optional logical app identity.
at	REAL	Required	Observation timestamp.
protocol	TEXT	Required	Observed TCP or UDP.
remote	TEXT	Required	Remote IP metadata; private.
port	INTEGER	Nullable	Remote port when available.
up	INTEGER	Required	Attributed upload IP bytes.
down	INTEGER	Required	Attributed download IP bytes.
mode	TEXT	Required	Observation provenance.

C Companion destinations fields

Table 37 *Companion destinations physical schema*

Field	SQLite type	Key or null	Meaning
app	TEXT	PK; see note	Logical app identity; key component.
remote	TEXT	PK 2; see note	Observed remote address; key component.
mode	TEXT	PK 3; see note	Provenance; key component.
at	REAL	Nullable	Last observation timestamp.

C Companion settings fields

Table 38 *Companion settings physical schema*

Field	SQLite type	Key or null	Meaning
key	TEXT	PK; see note	Preference identifier.
value	TEXT	Required	JSON encoded local preference.

SQLite schema note: a declared primary key in an ordinary rowid table does not always imply NOT NULL for non-integer columns. The current companion statements omit explicit NOT NULL on several text key fields; application validation supplies their intended values. A future schema migration should address this deliberately. Composite key numbers above indicate the declared key order, not separate keys.

C 2 Integrity and normalization

The research schema separates sessions, telemetry, windows, model metadata and analysis results to avoid repeating session identity and provenance in every value. Child rows use session foreign keys and observation-time uniqueness. Traffic windows enforce directional and protocol conservation, time ordering and consistency between partial, valid and reason fields. Model deletion is protected when anomaly results reference the model.

The lab schema trades deep relational metric querying for a bounded complete-result JSON document, with strict application validation. The companion normalizes executable identity and policy while aggregating usage by application, UTC day and provenance. Its SQL declares fewer foreign keys than its logical relationships; future strengthening must preserve the unassigned usage case and migration compatibility.

C 3 Migration and storage procedures

The lab application has three additive migrations: initial jobs, QueueGuard creation and singleton seeding. The original research migrations are preserved. The source launcher applies only the explicit standalone lab configuration. Companion SQLite uses user_version 1 and rejects unknown versions. A schema version should be changed only with a documented migration and backup path.

Retention limits rows and observation age, but deleting SQLite rows does not necessarily reduce the database file immediately. Storage maintenance must be planned separately from logical retention. Backup and cleanup procedures must avoid including secrets or private inventories in public project submissions.

Appendix D Test Execution and Acceptance Procedures

D 1 Repeatable commands

```
.\.venv-lab\Scripts\python.exe -m unittest discover -s tests/lab -p "test_*.py"
```

```
.\.venv-backend\Scripts\python.exe backend/manage.py test tests --  
settings=config.test_settings
```

```
.\.venv-backend\Scripts\python.exe backend/manage.py makemigrations --check --dry-run --  
settings=config.test_settings
```

```
.\.venv-lab\Scripts\python.exe scripts/verify_ai_lab.py
```

```
cd frontend
```

```
npm test
```

```
npm run lint
```

```
npm run typecheck
```

```
npm run build
```

```
npm run test:e2e
```

```
npm run build:public
```

Run commands with their intended environments and required configuration. Django test_settings is for isolated tests only. A test database is not a running product configuration. Build and browser checks should use a prepared production frontend when validating release behaviour.

D 2 Manual academic demonstration

1. State the problem and the SIMULATION boundary before showing metrics.
2. Start the local launcher and sign in without displaying any password or token.
3. Run the default scenario and show actual generation, training and evaluation stages.
4. Explain the whole-run split and show the number of unavailable test windows.
5. Compare Random Forest, heuristic rules and the dummy baseline using macro F1 and the confusion matrix.
6. Show the low anomaly recall and explain the threshold trade-off without changing test results.
7. Open a SHAP case and relate the contribution sum to the model output.
8. Use recorded playback and the truth overlay, explaining that neither is live traffic.
9. Download a report, cancel another job and reopen retained history after restart.
10. Conclude with the measured limits and the next external validation gate.

D 3 Remaining Windows verification

Use explicitly authorized, controlled applications and record the selected interface, executable identity and observation coverage. Compare independent controlled transfer bytes with attributed IP bytes while documenting layer and retransmission differences. Exercise multiple processes with the same executable, ambiguous sockets and traffic that remains unassigned.

Enable enforcement only for a suitable controlled application. Record separately whether the owned rule was applied and whether new traffic was actually blocked. Measure bytes beyond a quota threshold and the delay between observation, reconciliation and effective blocking. Verify temporary overrides, UTC resets, restart persistence and protection of critical executables.

Test normal shutdown cleanup and abnormal process termination followed by recovery. Inspect only NetSentinel-owned rules and retain evidence that unrelated rules remain unchanged. Defender scan testing must be explicit, handle unavailable or passive states and avoid claiming percentage progress that Windows does not expose. No malicious sample is required for the academic simulation demonstration.

Appendix E Evidence Register and Release Handoff

E 1 Source and generated evidence

The source repository is <https://github.com/hariharan4814/NetSentinel-upgraded>. The report's inspected branch is `codex/ai-lab-prototype` and the source checkpoint is `d0fbbb1`. The draft review is pull request 1 and is not merged. The report preparation changes documentation and generated document artifacts only.

The genuine default experiment is retained below `artifacts/lab/browser-qa-20261006/experiments/856e32e1-6c30-4b6e-97d2-fc35e877cdac`. Its `result.json` supplies the principal tables in Chapter 8. The directory name reflects the start of browser QA; the final cited experiment completed on 7 October 2026.

Table 39 *Retained evidence locations relative to the repository*

Artifact	Location and purpose
Final integration result	<code>artifacts/lab/integration-20261007-220124/verification.json</code>
Repeated seed study	<code>artifacts/lab/repeated-seed-study-v2/benchmark.json</code>
Default result	<code>browser-qa-20261006</code> experiment directory identified above
Desktop and mobile evidence	<code>artifacts/lab/desktop-verified.png</code> and <code>mobile-verified.png</code>
Application-generated PDF	<code>output/pdf/NetSentinel-AI-Lab-SIMULATION.pdf</code>
Reviewed source archive	<code>artifacts/release/NetSentinel-AI-Lab-source.zip</code>
Resource sources and hashes	<code>resources/ai-lab/manifest.json</code>
Durable execution history	<code>plan.md</code> and <code>YT.md</code>
Executable and API contract	<code>docs/AI_LAB_CONTRACT.md</code>

The dataset `SHA-256` is `de4c289d3f3916769ecdf0f32db44c494c40c9b589d7fd9876319da292c50fe6`. This digest identifies the generated dataset; it is not a digital signature or proof of authorship. Generated models, datasets and screenshots are intentionally excluded from normal source control and must be preserved separately for reproduction when moving machines.

E 2 Release classification

The local AI prototype is implemented and verified within the evidence scope recorded here. PostgreSQL acceptance and official external dataset acquisition remain blocked. The language-

model assistant, public AI showcase, drift and ablation work, production Windows acceptance and signing are deferred. A source archive is available, but it is not a signed installer.

The existing public connection site is <https://netsentinel-connect.hariharan4814.chatgpt.site>. Its recorded successful publication predates the completed AI laboratory. This report does not claim that /lab is publicly deployed or that the public site can control an installed Windows companion. Only the allowlisted public artifact is eligible for website publication.

E 3 License and attribution record

scikit-learn 1.9.0 is used under BSD-3-Clause and SHAP 0.51.0 under MIT, with retained license texts. The locked transitive numerical packages have their own license and notice obligations. The project uses Django, React, Next.js and PDF libraries as dependencies; those upstream algorithms and frameworks are credited rather than represented as original work.

The source release does not republish the downloaded research papers or optional dataset. Service terms for public measurement endpoints are separate from software licenses and must be rechecked before broader promotion. No project-wide license grant is inferred for the owner's source merely because its dependencies are open source.

Appendix F Glossary and Viva Notes

F 1 Glossary

Table 40 Project abbreviations and terms

Term	Meaning in this project
API	Application Programming Interface
ASN	Autonomous System Number
CSRF	Cross-Site Request Forgery
DFD	Data Flow Diagram
ER	Entity Relationship
IF	Isolation Forest deviation model
RF	Random Forest classifier
SHAP	SHapley Additive exPlanations
RSS	Resident Set Size of a process
UTC	Coordinated Universal Time
LIVE	Genuine observations from the stated observation point
SIMULATION	Generated metadata, explicitly separated from live evidence
REPLAY	Previously recorded compatible observations processed later
Provenance	Origin, mode, version and context needed to interpret data
Unscored	No valid model result for an unavailable or incomplete observation
Macro F1	Unweighted mean of the specified per-class F1 scores
Lease	Expiring worker authority to complete one claimed job

F 2 Questions for the project demonstration

Why use simulation? It permits repeatable experiments and privacy-preserving demonstrations without sending attack traffic. Why use two models? The classifier identifies generated families, while the anomaly detector measures deviation from benign training behaviour. Their tasks and metrics are different.

Why split whole runs? Windows from one run share generation context. A random window split can make test observations too similar to training observations. Why keep a dummy

baseline? It shows how much apparent accuracy comes from class frequency. Why is macro F1 useful? It gives each specified class equal weight rather than allowing routine traffic to dominate the summary.

Does SHAP prove the cause of a prediction? It explains the fitted model relative to a specified background and masking procedure, not a real-world cause. Does a low anomaly score prove safety? No; it means the observation did not exceed this model's threshold. Why report poor recall? Because the measured limitation is necessary for a scientifically valid conclusion and guides the next experiment.

Can the public website capture packets or block an application? No. Capture belongs to the separately installed native software, and privileged controls require the narrowly scoped broker. Can the simulation model be used for LIVE traffic immediately? No. Matching feature names do not establish measurement compatibility or live performance.

F 3 Submission completion fields

Replace the cover placeholders with the approved student, institution, supervisor and academic-year details. Add institution-specific certificate, declaration and signature pages only using the approved wording and genuine signatures. The technical chapters, tables and experiment findings should retain their stated scope and dates when the cover is completed.